

Dashnosis: bearing compound fault diagnosis overcoming scarcity of compound fault data and domain shift

Chao Peng^{1,†}, Yunhao Chen^{2,†}, Yixin Liu¹, Xihong Chen¹, Biao Wang^{1,3,*}, Ergude Bao^{1,*}, Qi Liu^{1,4,*}

¹ School of Software Engineering, Beijing Jiaotong University, No. 3 Shangyuan Residence, Haidian District, Beijing 100044, China

² Faculty of Animation, Arts & Design, Sheridan College Institute of Technology and Advanced Learning, 1430 Trafalgar Road Oakville, ON L6H 2L1, Canada

³ Collaborative Innovation Center of Railway Traffic Safety, Beijing Jiaotong University, No. 3 Shangyuan Residence, Haidian District, Beijing 100044, China

⁴ CRRC Academy, 9th Floor Building 5 Nord Center II, Fengtai Science and Technology Park, Fengtai District, Beijing 100070, China

[†] Equal contribution. * Corresponding authors: wbiao@bjtu.edu.cn; baoe@bjtu.edu.cn; lq@crrc.tech

Abstract. Compound bearing fault diagnosis is a critical issue in the monitoring of the condition of mechanical equipment. Compound bearing faults refer to the simultaneous occurrence of multiple types of damage in bearing components such as rolling elements and inner and outer races. They are typically identified by acquiring vibration signals and applying deep learning models. This task faces two major challenges. First, there is a scarcity of compound fault data, making it difficult to obtain a sufficient number of samples during the training stage. Second, there exists the domain shift between the training data and the actual operating conditions of target bearings. To address these two challenges in compound bearing fault diagnosis, we propose the Dashnosis method, which can be trained using single fault data from the source domain to achieve diagnosis of compound faults in the target domain. In Dashnosis, we propose the multi-class feature filtering decoder by integrating the Class-Specific Residual Attention (CSRA) multi-class classification framework and the Feature Attention Block (FAB) feature enhancement framework. Within this decoder, we further design the irrelevant feature filtering approach together with the branch freezing training approach to achieve compound fault diagnosis without compound fault training data. Meanwhile, in Dashnosis, we propose the feature transfer enhancement encoder based on the Transferable Vision Transformer (TVT) domain adaptation framework. Within this encoder, we design the progressive feature transfer approach and the time-frequency feature enhancement approach to solve the issue of domain shift. Experimental results on six diagnostic tasks using the BJTU-RAO dataset and HUSTbearind dataset demonstrate that Dashnosis can accurately identify compound bearing faults, verifying its effectiveness in practical cross operating condition fault diagnosis scenarios.

Keywords: compound fault diagnosis, scarcity of compound fault data, domain shift

1. Introduction

Compound bearing faults, involving the simultaneous occurrence of multiple single faults, are a common and challenging issue in rotating machinery. Compound bearing faults may occur during long-term operation, insufficient lubrication, impact loading, installation misalignment, local defect propagation, or coupled degradation of multiple bearing components. In practical rotating machinery, an initial local defect may change the contact state, load distribution, and dynamic response of the bearing, thereby accelerating the degradation of other components and eventually leading to multiple coexisting faults. Therefore, compound fault diagnosis is not only a multi-label classification problem, but also has practical significance for maintenance decision-making. Accurate diagnosis can help identify all fault components involved in the degradation process and provide more complete fault information for maintenance planning. If a compound fault is misdiagnosed as a single fault, maintenance actions may only address the dominant fault while hidden fault components remain untreated, which may further cause fault propagation, secondary damage, unexpected downtime, and potential safety risks. To address this, researchers typically acquire multi-modal signals under various conditions, using signal processing and data-driven methods to reveal fault characteristics. Although deep learning techniques such as convolutional neural networks and Transformer have advanced intelligent diagnosis, they still face limitations in generalization and accuracy under complex conditions, particularly due to the limited compound fault data and the domain shift between the source domain and the target domain [1–5].

(i) One major challenge in compound bearing fault diagnosis is scarcity of compound fault data. In practical industrial environments, compound bearing faults occur with relatively low probability and their formation processes are complex and uncontrollable, making it difficult to obtain a large number of representative compound fault samples under experimental conditions. In contrast, single faults are easier to collect, resulting in training datasets dominated by single fault samples while lacking real compound fault data. Scarcity of compound fault data prevents models from sufficiently learning the coupling characteristics among different faults, thereby degrading diagnostic accuracy and robustness. (ii) In addition, the domain shift between the source domain and the target domain constitutes another critical challenge in compound bearing fault diagnosis. Bearing vibration signals collected under different operating conditions, such as rotational speed, load and temperature, often exhibit substantial distribution differences. Even for the same type of fault, features may shift with changes in operating conditions. This inter domain distribution discrepancy causes models to perform well in the source domain, namely the training operating conditions, but experience a notable performance drop in the target domain, namely the testing operating conditions, making it difficult to achieve robust cross-domain capability.

To address challenge (i), Xu et al. introduced semantic information to model the semantic relationships between single faults and compound faults, thereby enabling the

recognition of unseen compound fault classes [6]. Meanwhile, Xu et al. constructed the label-level semantic generation model to characterize the compositional properties of compound faults [7]. Furthermore, in generalized zero-shot learning scenarios where seen and unseen classes coexist, Xu et al. used generative framework for compound fault diagnosis to improve generalization performance [8]. In addition, Xu et al. incorporated semantic alignment mechanisms to strengthen the consistency between the feature space and the semantic space, thereby enhancing the capability of generative zero-shot models to represent compound fault semantics [9]. Cen et al. proposed an intelligent compound fault diagnosis method based on generative generalized zero-shot learning, which generates compound fault semantics and features by training single fault samples and uses semantic matching to diagnosis zero-shot compound faults [10]. Wang et al. proposed a zero-shot model based on attribute embedding, which constructs attribute prototypes of single and compound faults and introduces a feature difference mapping activation function to enhance the discriminatory power of multi-class features, thereby achieving the diagnosis of compound faults without compound fault samples [11]. Yin et al. proposed a zero-shot compound fault diagnosis framework based on orthogonal high order semantics and cross level discriminative learning. This framework constructs a more separable high order semantic representation and uses cross level multi-stage optimization of discriminative capabilities, thereby achieving diagnosis in the case of zero compound faults [12]. Xiao et al. proposed a zero-shot learning method with hybrid semantic embedding, which constructs a new semantic space by combining manually defined and machine-extracted semantics, thereby generating single fault representations that can synthesize compound fault semantics to achieve zero-shot diagnosis of compound faults [13]. Shen et al. proposed a zero-sample compound fault diagnosis method based on semantic graph embedding and multi-stage fusion. By constructing semantic relationships from single faults to compound faults, the method can effectively identify unseen compound faults [14].

To address challenge (ii), Zhao et al. proposed a dual adversarial network to address cross-domain open-set fault diagnosis by jointly aligning shared feature distributions while separating unknown target-domain faults from known classes [15]. Ke et al. proposed an improved capsule network-based method for cross-domain compound fault diagnosis, which enhances feature representation and fault coupling modeling [16]. Wang et al. proposed a dynamic collaborative adversarial domain adaptation network that achieves unsupervised fault diagnosis by adaptively coordinating multiple adversarial objectives to align source and target domain features distributions [17]. He et al. proposed a time-frequency self-contrastive learning framework for cross-domain compound fault diagnosis, enabling effective cross-domain alignment by learning domain-invariant and discriminative joint time-frequency representations [18]. Pan et al. addressed cross-domain fault diagnosis in open-set scenarios by enhancing discriminative feature learning via supervised contrastive learning and achieving effective source-target domain alignment through a complementary weighted dual adversarial network [19]. Xia et al. addressed cross-domain compound fault diagnosis under limited single-fault

samples by employing fault semantic knowledge transfer learning to transfer semantic relationships across-domains and enable effective recognition of compound faults [20]. Zhou et al. proposed an uncertainty-aware digital twin-assisted domain adaptation framework to address cross-domain compound fault diagnosis, effectively mitigating cross-domain discrepancies and improving diagnostic performance with incomplete supervision [21]. Li et al. introduced a multi-kernel weighted joint domain adaptation approach to align marginal and conditional distributions for cross-condition compound fault diagnosis [22].

On the other hand, in the field of image processing, extensive researches have been conducted on multi-class classification and cross-domain transfer learning, resulting in relatively mature technical frameworks. There are the following methods. Transferable Vision Transformer (TVT) is used to achieve effective domain transfer (257 cites by March 2026) [23]. Its design centrally focuses on incorporating learnable transferable weights within the attention weights. Class-Specific Residual Attention (CSRA) can perform multi-class classification only using single class training samples (249 cites by March 2026) [24]. It is designed as a multi-branch framework, where each branch is dedicated to handling a single corresponding class. Feature Attention Block (FAB) is designed for feature enhancement (401 cites by March 2026) [25]. It first performs convolution in the channel dimension and then conducts convolution in the spatial dimension.

Nevertheless, compound fault diagnosis differs fundamentally from conventional image classification tasks. In compound fault samples, features of different classes exhibit strong coupling and discriminative differences among bearing fault characteristics are often subtle, making them difficult for traditional deep learning models to capture. Based on these characteristics, we propose the Dashnosis method, which can be trained using single fault data from the source domain to achieve diagnosis of compound faults in the target domain. Specifically, Dashnosis first constructs time domain features from raw vibration signals and frequency domain features using Short-Time Fourier Transform (STFT). These features are then processed by the feature transfer enhancement encoder to extract domain-invariant features, thereby mitigating domain discrepancies across operating conditions. The encoded time-frequency features are subsequently fused and fed into the multi-class feature filtering decoder, where class-specific branches selectively preserve fault related information while suppressing irrelevant interference. Finally, independent class classifiers generate diagnostic results for each fault category. (1) In Dashnosis, we propose the multi-class feature filtering decoder based on irrelevant feature filtering and branch freezing training integrating the core ideas of the CSRA framework and the FAB framework. Under only single class sample condition, this decoder can extract class relevant features from the complex space while suppressing irrelevant interference, thus enabling compound fault classification without any compound fault training samples. We design two key approaches. The first is the irrelevant feature filtering approach, which uses a joint channel and spatial attention structure to enhance class relevant channels along the channel dimension and

enhance features of key regions along the spatial dimension, thus achieving effective decoupling and enhancement of class relevant features. The second is the branch freezing training approach, during training, in which only the class branch corresponding to the current single class sample is activated, while the parameters of the remaining branches remain frozen; during inference, all branches are activated. (2) In Dashnosis, we propose the feature transfer enhancement encoder based on progressive feature transfer and time-frequency feature enhancement improving the TVT framework. Therefore, this encoder can extract domain invariant features both in the frequency domain and in the time domain, while also obtaining enhanced features within the frequency domain. In this encoder, we design the progressive feature transfer approach, where the decoder progressively transfers and enhances features from earlier layers to later layers. Meanwhile, we design the time-frequency feature enhancement approach, frequency-domain features serve as the query, while time-domain features serve as the key and value. It should be noted that Dashnosis does not physically decompose compound vibration signals into independent single-fault waveforms. Instead, it performs feature-space decoupling guided by compound fault characteristics. Since different fault sources may generate overlapping characteristic frequencies, harmonics, impulses and amplitude modulations, the key objective is to extract class relevant fault responses from coupled time-frequency representations. The TVT-inspired encoder enhances transferable fault related features across operating conditions. The CSRA-inspired class-specific branches guide different branches to focus on different fault classes, while the FAB-inspired channel and spatial filtering suppress irrelevant responses and enhance class relevant regions. Therefore, Dashnosis identifies multiple coexisting fault components through class relevant feature filtering rather than explicit physical signal separation.

Figure 1 illustrates the techniques used in Dashnosis. The main contributions of this paper can be summarized as follows.

- 1) We propose Dashnosis, a novel framework for cross-domain compound bearing fault diagnosis. Dashnosis enables compound fault recognition in the target domain using only single fault samples from the source domain, addressing the challenging scenario of zero compound fault training samples.
- 2) In Dashnosis, we propose the Multi-class Feature Filtering Decoder to address the scarcity of compound fault data. By introducing the irrelevant feature filtering approach and the branch-freezing training approach, the decoder can selectively extract class-specific fault features while suppressing irrelevant information, thereby achieving compound fault diagnosis without compound fault training samples.
- 3) In Dashnosis, we propose the Feature Transfer Enhancement Encoder to mitigate domain discrepancies across operating conditions. The encoder incorporates a progressive feature transfer approach and a time-frequency feature enhancement approach to learn discriminative and domain-invariant fault features for cross-domain diagnosis.
- 4) Extensive experiments under multiple cross-domain operating conditions demon-

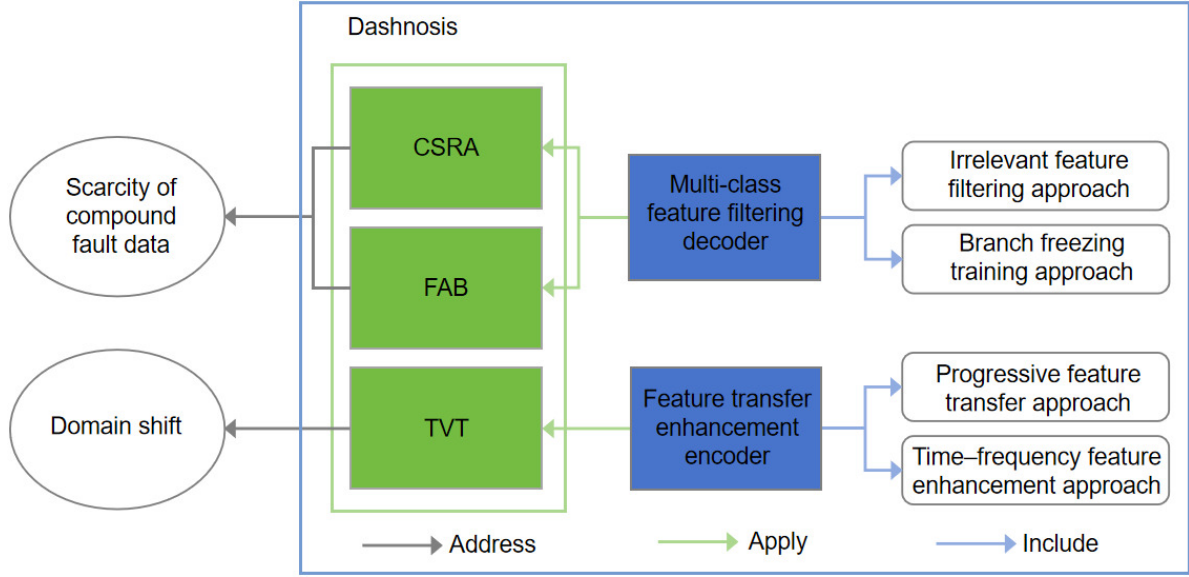


Figure 1: Technologies used in Dashnosis.

strate the effectiveness, robustness, and generalization capability of the proposed Dashnosis framework.

2. Preliminaries

2.1. CSRA method

The CSRA method [23] uses the shared feature extraction backbone network and class-specific attention branches for image multi-class classification. Specifically, for the extracted feature $f \in \mathbb{R}^{N \times D}$, K independent 1×1 convolutions are applied to perform class relevant feature selection, where the k -th convolution kernel is dedicated to extracting feature for the k -th class ($1 \leq k \leq K$), producing the attention weight c_k for that class.

$$c_k = \text{conv}_k(f) \quad (1)$$

Subsequently, for each class attention weight c_k global average pooling and global max pooling are applied, the results are fused using a learnable weight α . To enable element-wise multiplication with f , the fused result is projected to expand its dimensions to $N \times D$ and then multiplied with f to obtain the class relevant feature f_k for the k -th class, as shown in equation (4).

$$a_k = \text{averagePooling}(c_k) \quad (2)$$

$$m_k = \text{maxPooling}(a_k) \quad (3)$$

$$f_k = \text{projection}(a_k + \alpha m_k) f \quad (4)$$

The final classification is composed of K independent binary classifiers. For each class relevant feature f_k , the probability of belonging to class k is calculated using equation (5).

$$p_k = \text{sigmoid}(W_k f_k + b_k) \quad (5)$$

2.2. TVT method

The TVT method [24] achieves domain transfer by introducing transferable weights into the attention weights. For an input feature $f \in \mathbb{R}^{N \times D}$, designs a domain discriminator composed of multiple linear layers, which computes a domain score for f . The feature matrix f is fed into the discriminator and transformed through multiple linear layers to obtain the domain score r , as shown in equation (6).

$$r = W_2(W_1 f + b_1) + b_2 \quad (6)$$

Here, W_1 , b_1 , W_2 and b_2 denote the weights and biases of the corresponding linear layers, respectively. Then, to match the dimensionality of the $N \times N$ attention weight matrix, r is broadcast to obtain the domain weight matrix $M_d \in \mathbb{R}^{N \times N}$.

$$M_d = \text{broadcasting}(r) \quad (7)$$

Subsequently, the domain weight matrix M_d is incorporated into the attention weight of the feature f and multiplied with the original feature f , as shown in equation (8).

$$f' = \text{softmax}\left(\frac{f f^T}{\sqrt{d}}\right) M_d f \quad (8)$$

2.3. FAB method

The FAB method [25] achieves irrelevant feature selection through the channel and spatial dimensions. Specifically, given the feature $f \in \mathbb{R}^{N \times D}$, feature extraction is first conducted along the N dimension, referred to as the channel filtering. The channel filtering obtains channel level global feature g through average pooling, then generates channel weight $c \in \mathbb{R}^{N \times 1}$ via two convolutional layers and nonlinear activation functions, as shown in equation (10).

$$g = \text{averagePooling}(f) \quad (9)$$

$$c = \sigma(\text{conv}_N(\delta(\text{conv}_N(g)))) \quad (10)$$

Here, δ denotes the relu function and σ denotes the sigmoid function. To enable element wise weighting, c is projected to expand its dimensionality to $N \times D$, then multiplied with the feature f to perform channel wise feature filtering, as shown in equation (11).

$$f' = \text{projection}(c) \odot f \quad (11)$$

Subsequently, feature extraction is performed along the D dimension, referred to as the spatial filtering. Through two convolutional layers and nonlinear activation functions, spatial weight $s \in \mathbb{R}^{1 \times D}$ is generated, as described in equation (12). To enable element

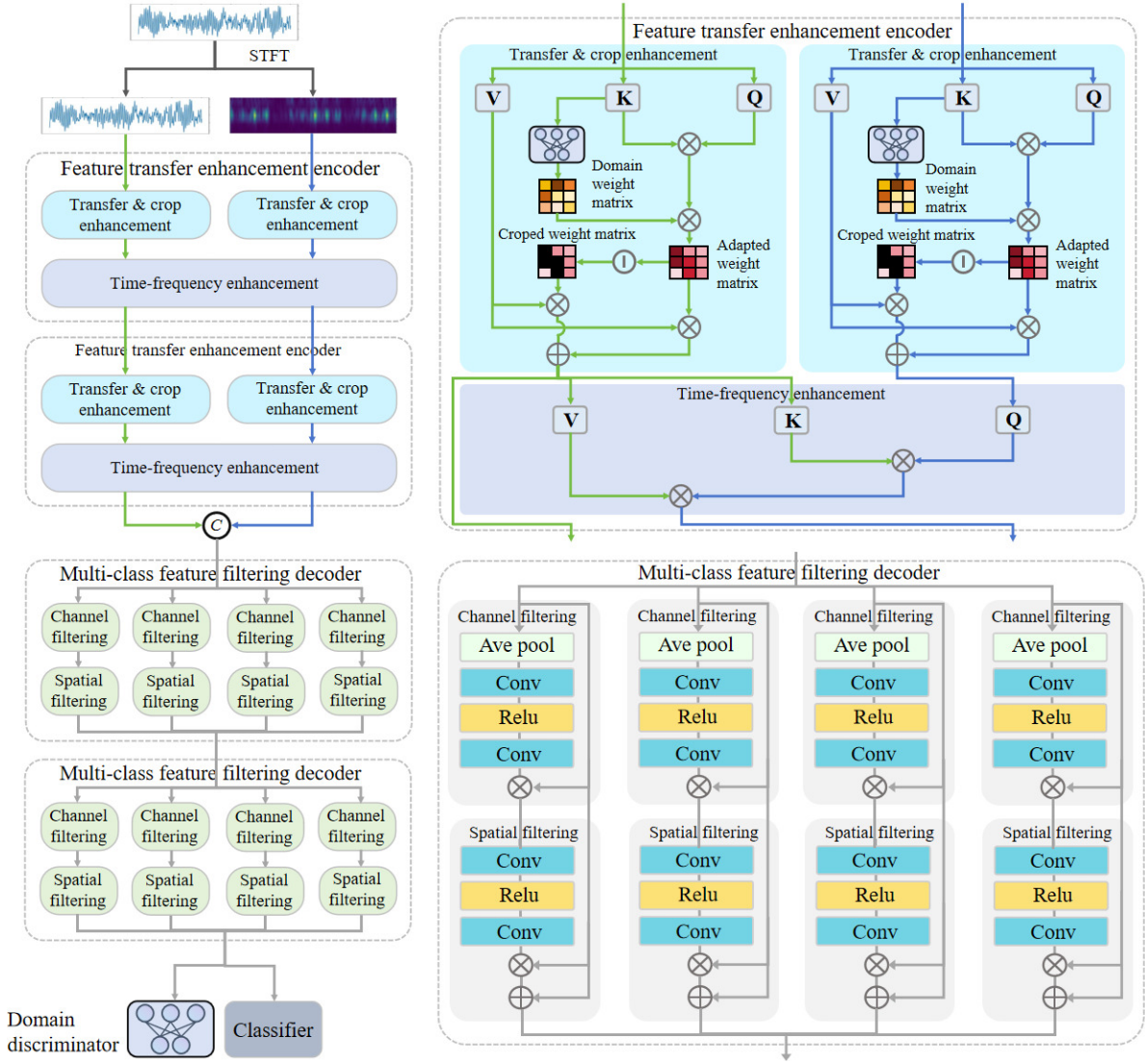


Figure 2: Dashnosis framework diagram.

wise weighting, s is projected to expand its dimensionality to $N \times D$ and then multiplied with f' to obtain the final filtered feature f'' , as shown in equation (13).

$$s = \sigma(\text{conv}_D(\delta(\text{conv}_D(f')))) \quad (12)$$

$$f'' = f' \odot \text{projection}(s) \quad (13)$$

3. Method

3.1. Overview

Dashnosis takes the time domain data and the frequency domain data as dual inputs and outputs multi-class fault types. The overall architecture is illustrated in Figure 2.

- (1) Dashnosis converts the time domain data into frequency domain data via STFT

and applies linear projection to both time domain and frequency domain data to obtain their corresponding embeddings.

- (2) The time domain embeddings and the frequency domain embeddings are fed into multiple encoding layers. In each encoding layer, through the progressive feature transfer approach, domain invariant features in both time domain and frequency domain are progressively extracted. By the time-frequency feature enhancement approach, the enhanced frequency domain feature is obtained. After the outputs of the final encoding layer, the two types of features are concatenated to form a fused feature.
- (3) The fused feature is input into multiple decoding layers. Within each decoder layer, multiple class feature filtering branches are constructed, with each branch corresponding to a fault class. This ensures that each class branch selectively extracts features related to its corresponding class, avoiding inter class feature interference and achieving compound feature decoupling. The branch freezing training approach is used. During training, where only the class branch corresponding to the class of the current sample is activated, while the remaining branches remain frozen; during inference, all branches are activated.
- (4) The filtered features from each class branch are then fed into their corresponding independent classifiers to output the classification results for each fault class.

3.2. Data preprocessing

For the input raw data $X \in \mathbb{R}^S$, where S denotes the length of the original signal, it is treated as time domain data X_t . X_t is divided into N non overlapping patches, the sequence of time domain patches are obtained as $X_t = \{x_t^1, \dots, x_t^i, \dots, x_t^N\}$. Subsequently, each patch is linearly projected to obtain the time domain embedding, as shown in equation (14).

$$z_t^i = W_t x_t^i + b_t \quad (14)$$

Here, $W_t \in \mathbb{R}^{P \times D}$ is a projection matrix, D denotes the embedding dimension. After projection, the sequence of time domain embeddings are obtained as $Z_t = \{z_t^1, \dots, z_t^i, \dots, z_t^N\} \in \mathbb{R}^{N \times D}$.

In the following, the time domain data $X_t \in \mathbb{R}^S$ is transformed using STFT to obtain the frequency domain data $X_f \in \mathbb{R}^S$. Specifically, X_f is divided into N non overlapping patches, the sequence of frequency domain patches are obtained as $X_f = \{x_f^1, \dots, x_f^i, \dots, x_f^N\}$, each frequency domain embedding is obtained through linear projection, as shown in equation (15).

$$z_f^i = W_f x_f^i + b_f \quad (15)$$

The sequence of frequency domain embeddings are obtained as $Z_f = \{z_f^1, \dots, z_f^i, \dots, z_f^N\} \in \mathbb{R}^{N \times D}$.

3.3. Feature transfer enhancement encoder

For the encoding of time domain and frequency domain embeddings, L encoder layers E_l ($l = 1, 2, \dots, L$) are used. Each layer consists of progressive feature transfer approach and time-frequency feature enhancement approach. The input to each encoder layer is the time domain embedding Z_t^{l-1} and the frequency domain embedding Z_f^{l-1} output from the former layer, the layer outputs the enhanced features Z_t^l and Z_f^l . When $l = 0$, $Z_t^0 = Z_t$ and $Z_f^0 = Z_f$. The input-output relationships of the remaining layers are defined in equation (16).

$$Z_t^l = E_t^l(Z_t^{l-1}), \quad Z_f^l = E_f^l(Z_f^{l-1}) \quad (16)$$

After processing through the L encoder layers, the outputs $Z_t^L \in \mathbb{R}^{N \times D}$ and $Z_f^L \in \mathbb{R}^{N \times D}$ are obtained. These two features are concatenated along the feature dimension to form the fused feature $F \in \mathbb{R}^{N \times 2D}$.

$$F = \text{concat}(Z_t^L, Z_f^L) \quad (17)$$

In the following, the data flow within the l -th encoder layer is described in detail. The inputs to this layer are $Z_t = Z_t^{l-1}$ and $Z_f = Z_f^{l-1}$ and the outputs are $Z_t' = Z_t^l$ and $Z_f' = Z_f^l$.

3.3.1. Progressive feature transfer approach

The encoder extracts domain-invariant feature by weighting the attention coefficients with domain scores obtained from a domain discriminator. The specific procedure is as follows. Taking the time domain embeddings as an example, in the transfer and crop enhancement, the input is projected to obtain Q_t , K_t , V_t . The key K_t is then fed into the domain discriminator to produce the domain score r , as defined in equation (19).

$$Q_t = Z_t W_t^Q, \quad K_t = Z_t W_t^K, \quad V_t = Z_t W_t^V \quad (18)$$

$$r = W_2(W_1 K_t + b_1) + b_2 \quad (19)$$

$$M_d = \text{broadcasting}(r) \quad (20)$$

Here, $W_t^Q, W_t^K, W_t^V \in \mathbb{R}^{D \times D}$ denote the query, key and value projection matrices. Subsequently, the domain weight matrix M_d is applied to reweight the attention weight of the time domain embeddings, yielding the adaptive attention weight matrix M_a , as formulated in equation (21).

$$M_a = \text{softmax}\left(\frac{Q_t K_t^T}{\sqrt{d}}\right) M_d \quad (21)$$

On this basis, high-weight regions are cropped, while low-weight regions are retained. The original weight matrix and the cropped weight matrix are multiplied with the feature and then fused. Specifically, for the adaptive attention weight matrix M_a , the cropped threshold θ is defined, weight values exceeding the threshold are set to zero, while the remaining weight values are preserved, yielding the cropped attention weight

matrix M_c . Finally, both M_a and M_c are applied to the value V_t and summed, as shown in equation (22) and equation (23), resulting in the time domain branch feature Z'_t .

$$M_c(x, y) = \begin{cases} 0, & M_a(x, y) \geq \theta \\ M_a(x, y), & M_a(x, y) < \theta \end{cases} \quad x, y = 1, 2, \dots, N \quad (22)$$

$$Z'_t = M_a V_t + M_c V_t \quad (23)$$

Similarly, the same operations are applied to the frequency domain branch on the frequency domain embeddings to obtain the corresponding branch feature Z'_f .

3.3.2. Time-frequency feature enhancement approach

The time-frequency enhancement is used to facilitate information interaction between the time domain feature and the frequency domain feature. Specifically, the frequency domain feature Z'_f is used as the query, while the time domain feature Z'_t serves as the key and value, producing the enhanced frequency domain feature Z''_f , as shown in equation (24).

$$Z''_f = \text{multiHeadCross}(Z'_f, Z'_t, Z'_t) \quad (24)$$

3.4. Multi-class feature filtering decoder

The decoder consists of H stacked decoding layers D_h ($h = 1, 2, \dots, H$). Each decoding layer contains K parallel feature filtering branches, with each branch corresponding to a fault class. Within each branch, class relevant features are selected through irrelevant feature filtering approach. The input to each decoder layer is the feature F^{h-1} output from the former layer, this layer outputs the feature F^h . When $h = 0$, $F^0 = F$. The input-output relationships of the remaining layers are defined in equation (25).

$$F^h = D^h(F^{h-1}) \quad (25)$$

After processing through the H decoding layers, the final output F^H is obtained. In the following, the data flow of the k -th branch in the h -th decoding layer is described in detail. The input of this branch is $F_k = F_k^{h-1}$, the output is $F''_k = F_k^h$.

3.4.1. Irrelevant feature filtering approach

First, go through channel filtering to enhance class relevant channels. Specifically, average pooling is applied to the input feature F_k to obtain the global feature g , which is then passed through two convolutional layers and nonlinear activation functions to generate the channel weight $c \in \mathbb{R}^{N \times 1}$, as shown in equation (27).

$$g = \text{averagePooling}(F_k) \quad (26)$$

$$c = \sigma(\text{conv}_N(\delta(\text{conv}_N(g)))) \quad (27)$$

Here, δ denotes the relu function and σ denotes the sigmoid function. To enable element wise weighting, c is projected to expand its dimensionality to $N \times D$, then multiplied

with the feature F_k to perform feature filtering along the channel dimension N , as shown in equation (28).

$$F'_k = \text{projection}(c) \odot F_k \quad (28)$$

In the following, after the spatial filtering, enhance the weak feature of spatial dimensions. Specifically, F'_k is passed through two convolutional layers with nonlinear activation functions to generate the spatial weight $s \in \mathbb{R}^{1 \times D}$, as defined in equation (29). To enable element-wise weighting, s is projected and expanded to $N \times D$, then multiplied with F'_k , finally combined with the original input F_k via a residual connection to obtain the final filtered output F''_k , as shown in equation (30).

$$s = \sigma(\text{conv}_D(\delta(\text{conv}_D(F'_k)))) \quad (29)$$

$$F''_k = F'_k \odot \text{projection}(s) + F_k \quad (30)$$

3.4.2. Branch freezing training approach

Dashnosis designs the branch freezing training approach in the multi-class feature filtering decoder. Specifically, during training, only the branch corresponding to the class of the current sample is activated, while all other branches are frozen; during inference, all branches are activated. In each forward and backward pass during training, when the class of a training sample is k , the decoder activates only the k -th branch B_k , computes its classification loss and domain loss and updates the parameters of this branch. The remaining $K - 1$ branches $B_1, \dots, B_{k-1}, B_{k+1}, \dots, B_K$ are frozen and do not participate in parameter update ($1 \leq k \leq K$). The detailed training procedure is illustrated in Section 3.6.

3.5. Multi-class classifier

Dashnosis uses the fully connected network with an identical architecture but independent parameters as the classifier for each decoder branch. Specifically, the k -th classifier takes the corresponding branch filtered feature F_k^H as input and outputs the predicted probability $\hat{y}_k$ of the k -th fault class ($1 \leq k \leq K$), as shown in equation (31).

$$\hat{y}_k = \text{sigmoid}(WF_k^H + b) \quad (31)$$

The outputs of the K classifiers jointly form the final prediction $\hat{y} = [\hat{y}_1, \dots, \hat{y}_k, \dots, \hat{y}_K]$.

3.6. Training process

The loss function of Dashnosis consists of four core components, the classification loss, the time domain domain adaptation loss, the frequency domain domain adaptation loss and the global domain adaptation loss. Among them, the classification loss is formulated using binary cross-entropy. For each training batch, Dashnosis first aligns the classes with the decoder branches by partitioning the samples in the batch into K class-specific subsets according to their classes, where all samples in each subset belong to the same class. For each subset, only the branch corresponding to its class is activated while

the remaining branches are frozen, the feature is then fed into the activated branch. Specifically, for the k -th class subset in the current batch containing M samples, branch B_k is activated, its predicted outputs $\hat{y}_k$ are used to compute the classification loss, as defined in equation (32). The frozen branches also produce predictions, but these outputs are excluded from loss computation.

$$L_c = -\frac{1}{M} \sum_{i=1}^M [y_k^i \log(\hat{y}_k^i) + (1 - y_k^i) \log(1 - \hat{y}_k^i)] \quad (32)$$

In addition, for the computation of the domain adaptation losses, the initial domain loss is set to $L_p = 0$ when $l = 0$, the final domain loss L_p is obtained by accumulating the losses over L encoder layers. Specifically, for the time domain feature Z_t^l output by the l -th encoder layer, they are fed into the domain discriminator to obtain the domain score $\hat{r}$ based on which the time domain domain adaptation loss L_t^l is computed, as defined in equation (33).

$$L_t^l = -\frac{1}{M} \sum_{i=1}^M [r^i \log(\hat{r}^i) + (1 - r^i) \log(1 - \hat{r}^i)] \quad (33)$$

Here, r^i denotes the truth domain label of the i -th sample. Similarly, for the frequency domain feature Z_f^l output by the l -th encoder layer, they are fed into the domain discriminator to obtain the score $\hat{r}$, the frequency domain domain adaptation loss L_f^l is computed according to equation (34).

$$L_f^l = -\frac{1}{M} \sum_{i=1}^M [r^i \log(\hat{r}^i) + (1 - r^i) \log(1 - \hat{r}^i)] \quad (34)$$

At the final encoder layer, the time domain and frequency domain domain adaptation losses from all preceding layers are aggregated to obtain the final domain loss L_p .

$$L_p = \frac{1}{L} \sum_{l=1}^L (L_t^l + L_f^l) \quad (35)$$

The global domain loss is computed based on the output of the global domain discriminator. Specifically, the global domain discriminator takes the output of the final decoder layer F^H as input, produces the global domain score $\hat{r}_g$, the global domain loss L_g is calculated as defined in equation (36).

$$L_g = -\frac{1}{M} \sum_{i=1}^M [r^i \log(\hat{r}_g^i) + (1 - r^i) \log(1 - \hat{r}_g^i)] \quad (36)$$

The total loss function L_{total} is defined as a weighted sum of the losses described above, as shown in equation (37).

$$L_{\text{total}} = L_c + \lambda(L_p + L_g) \quad (37)$$

Here, λ is the hyperparameter of domain loss. The training procedure of Dashnosis is summarized in Algorithm 1.

Algorithm 1 Training process of Dashnosis

```

1: Input: raw data  $X$ , where the classification label of the  $X^i$  is  $y^i$  and the domain
   label is  $r^i$ 
2: Output: Trained parameters of Feature transfer enhancement encoder  $\theta_F$ , Multi-
   class feature filtering decoder  $\theta_M$ , Domain discriminator  $\theta_D$ , Global domain
   discriminator  $\theta_G$  and Classifier  $\theta_C$ 
3: Initialize parameters  $\theta_F, \theta_M, \theta_D, \theta_G, \theta_C$ 
4: for  $e = 1$  to  $n$  epochs do
5:   for each training batch  $\{x^i, y^i, r^i\}_{i=1}^b$  do
6:      $X_t \leftarrow X^i, X_f \leftarrow \text{STFT}(X^i)$ 
7:      $Z_t^0 \leftarrow \text{PatchEmbedding}(X_t), Z_f^0 \leftarrow \text{PatchEmbedding}(X_f)$ 
8:     for  $l = 1$  to  $L$  do
9:        $Z_t^l, Z_f^l \leftarrow \text{FeatureTransferEnhancementEncoder}(Z_t^{l-1}, Z_f^{l-1}; \theta_F)$ 
10:       $\hat{r} \leftarrow \text{DomainDiscriminator}(Z_t^{l-1}, Z_f^{l-1}; \theta_D)$ 
11:    end for
12:    Using equation (35), calculate the domain loss  $L_p$ 
13:     $F \leftarrow \text{concat}(Z_t^L, Z_f^L)$ 
14:    for  $k = 1$  to  $K$  do
15:      Freeze the other branches and only keep the  $k$  branch for training
16:       $F_k^H \leftarrow \text{MultiClassFeatureFilteringDecoders}(F, k; \theta_M)$ 
17:       $\hat{y}_k \leftarrow \text{Classifier}(F_k^H; \theta_C)$ 
18:    end for
19:    Using equation (32), calculate the classification loss  $L_c$ 
20:     $\hat{r}_g \leftarrow \text{GlobalDomainDiscriminator}(F^H; \theta_G)$ 
21:    Using equation (36), calculate the global domain loss  $L_g$ 
22:    Using equation (37), calculate the total loss  $L_{\text{total}}$ 
23:    Update  $\theta_F, \theta_M, \theta_D, \theta_G, \theta_C$  via backpropagation
24:  end for
25: end for

```

4. Case study I: performance evaluation based on BJTU-RAO dataset*4.1. The Introduction of the BJTU-RAO dataset*

To evaluate the cross operating condition compound fault diagnosis performance of the proposed Dashnosis method, vibration signals from the left axle box in the BJTU-RAO dataset are used in this study [26], published by our group. The dataset was collected on a subway bogie simulation test platform, which consists of four main components, a traction motor, a gearbox, a left axle box and a right axle box. The platform allows controllable variations in motor speed and lateral load. The health condition of the left axle box is categorized into seven classes, normal condition (NC), inner-race fault (IF), outer-race fault (OF), rolling-element fault (RF), inner-race and outer-race fault

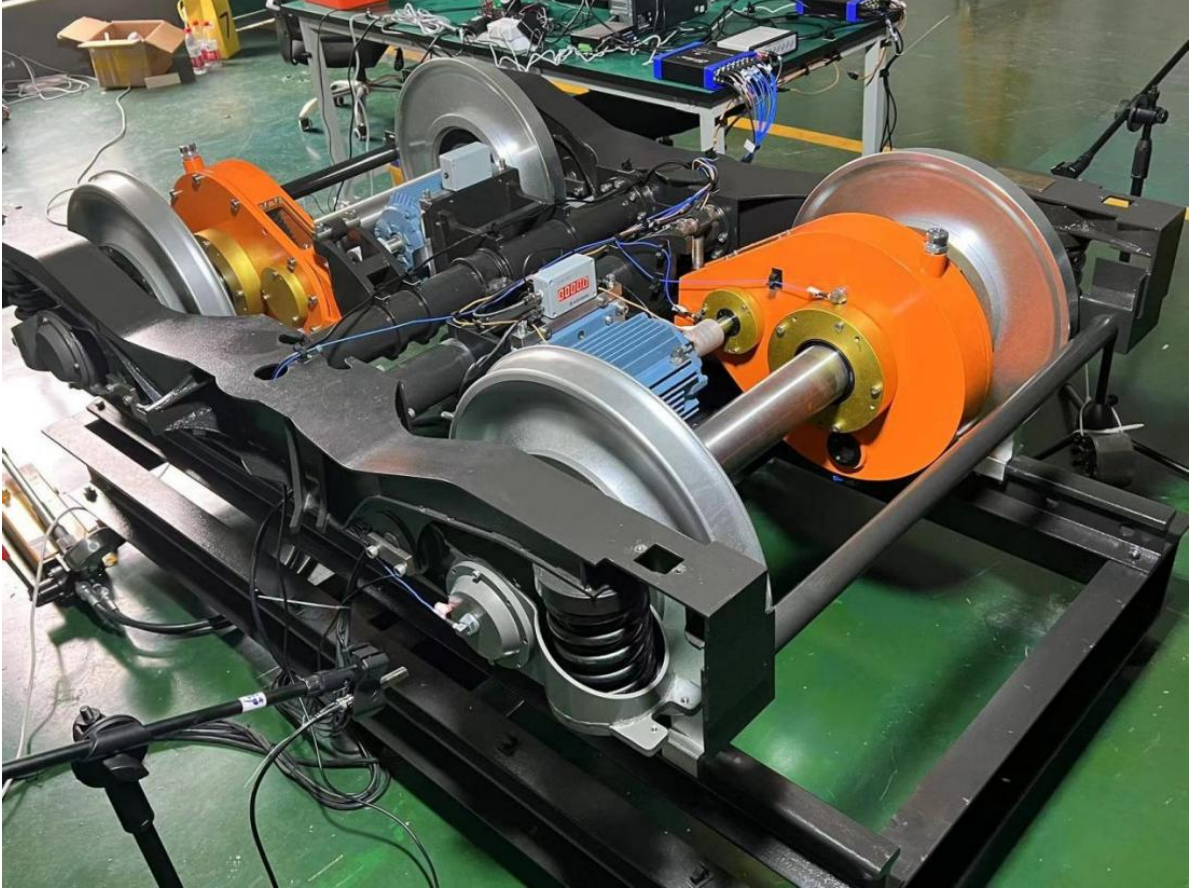


Figure 3: BJTU-RAO dataset experimental platform.

Table 1: Details of the BJTU-RAO dataset for training and testing.

Dataset	Working condition	Fault type for training	Fault type for testing	Samples
C1	20Hz/0kN	NC, IF, OF, RF	—	900
C2	40Hz/0kN	NC, IF, OF, RF	NC, IF, OF, RF, IOF, ORF, IORF	1575
C3	20Hz/-10kN	NC, IF, OF, RF	NC, IF, OF, RF, IOF, ORF, IORF	1575
C4	40Hz/-10kN	NC, IF, OF, RF	NC, IF, OF, RF, IOF, ORF, IORF	1575

(IOF), outer-race and rolling-element fault (ORF) and inner-race, outer-race and rolling-element fault (IORF). Experimental operating conditions are jointly determined by motor speed and lateral load, with motor speeds set to 20Hz and 40Hz and lateral loads set to 0kN and -10kN. An image of the experimental platform is shown in Figure 3.

Data were collected under four typical operating conditions, 20Hz/0kN, 40Hz/0kN, 20Hz/-10kN and 40Hz/-10kN, forming four datasets denoted C1, C2, C3 and C4. Each fault class contains 200 samples and includes all seven fault classes (NC, IF, OF, RF, IOF, ORF and IORF), as summarized in Table 1. The method is trained using NC, IF, OF and RF samples from one dataset and then tested on all seven classes from another

Table 2: BJTU-RAO dataset used in this study.

Task	Source domain	Target domain
C1→C2	C1	C2
C1→C3	C1	C3
C1→C4	C1	C4
C2→C3	C2	C3
C2→C4	C2	C4
C3→C4	C3	C4

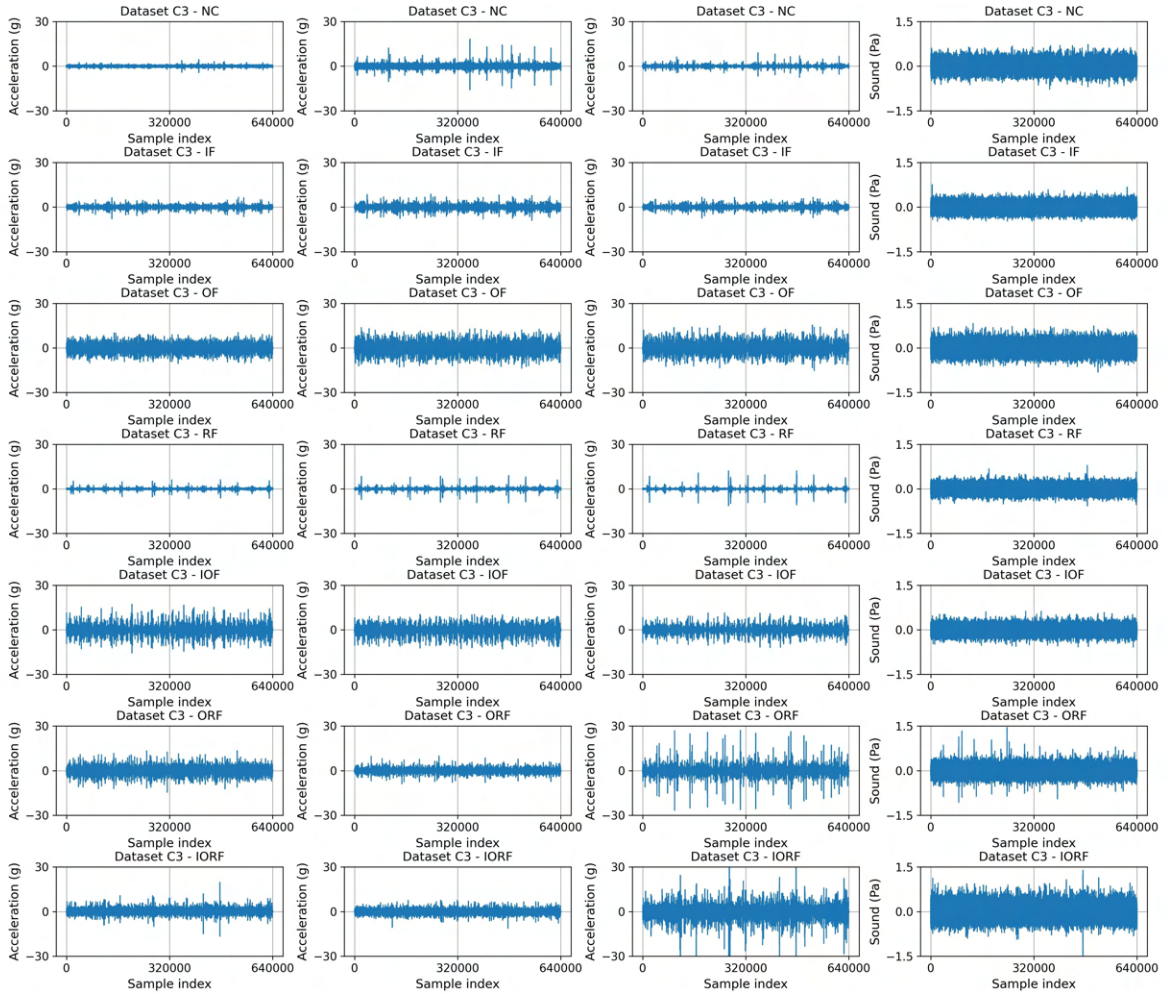


Figure 4: Signals of each fault in dataset C3 on BJTU-RAO dataset.

dataset. For example, C1→C2 denotes the transfer task with C1 as the source domain and C2 as the target domain. Based on datasets C1, C2, C3 and C4, six different transfer tasks are constructed to evaluate the performance of various methods, as listed in Table 2. Figure 4 illustrates the signals of different fault types in dataset C3.

Table 3: Hyperparameter settings.

Hyperparameter	Symbol	Value
Signal length	S	1280
Patch number	N	8
Embedding dimension	D	160
Encoder layer	L	3
Decoder layer	H	3
Cropped threshold	θ	0.5
Hyperparameter of domain loss	λ	0.5
Epoch number	n	50
Batch size	b	64

4.2. Hyperparameter setting

To ensure stable training and fair comparison of the Dashnosis method in cross-operational condition scenarios, unified hyperparameters are used for both the encoder and decoder. The main parameter configurations are as follows. Each vibration signal sample has a length of $S = 1280$ and is divided into $N = 8$ non-overlapping patches. Each patch is linearly projected into an embedding $D = 160$, which is then fed into the multi-layer encoders for the time domain and the frequency domain. Frequency domain data is obtained via STFT with a window size of 32 and an overlap ratio of 0.5. The encoder consists of $L = 3$ layers, the dimensions of the query, the key and the value projection matrices are all set to 160. The hyperparameter of domain loss is set to $\lambda = 0.5$. The cropped threshold is set to $\theta = 0.5$. The decoder contains $H = 3$ layers. The Adam optimizer is used with an initial learning rate of 1×10^{-4} , the training epochs $n = 50$ and the batch size $b = 64$. The detailed hyperparameters and their values are summarized in Table 3.

4.3. Test results on different tasks

Within the evaluation framework of six tasks, we conduct a comprehensive assessment of the fault diagnosis performance of Dashnosis. Table 4 presents the accuracy of different fault types for each task, while Table A1 shows the corresponding metrics such as recall, precision, and F1 score. The results showed that the method achieved high diagnostic accuracy for all fault types across every task, demonstrating stable and reliable identification capabilities.

Further visual analysis is presented in Figure 6, which illustrates the predicted probability distribution across different fault types for all test samples. In the figure, the intervals of 0–199, 200–399, 400–599, 600–799, 800–999, 1000–1199 and 1200–1399 correspond to the NC, IF, OF, RF, IOF, ORF and IORF fault types. For the three types of compound faults, the method as a whole can provide very good classification results.

Table 4: Dashnosis accuracies of each fault on BJTU-RAO dataset (%).

Task	NC	IF	OF	RF	IOF	ORF	IORF	Average
C1→C2	100.00%	100.00%	100.00%	91.11%	97.78%	100.00%	99.56%	98.35%
C1→C3	100.00%	99.11%	100.00%	96.44%	100.00%	100.00%	97.33%	98.98%
C1→C4	100.00%	100.00%	100.00%	96.44%	99.56%	99.11%	100.00%	99.30%
C2→C3	100.00%	99.56%	100.00%	94.67%	100.00%	97.33%	95.11%	98.10%
C2→C4	100.00%	100.00%	99.56%	98.67%	98.67%	100.00%	98.67%	99.37%
C3→C4	100.00%	100.00%	99.56%	96.00%	99.56%	96.44%	99.56%	98.73%

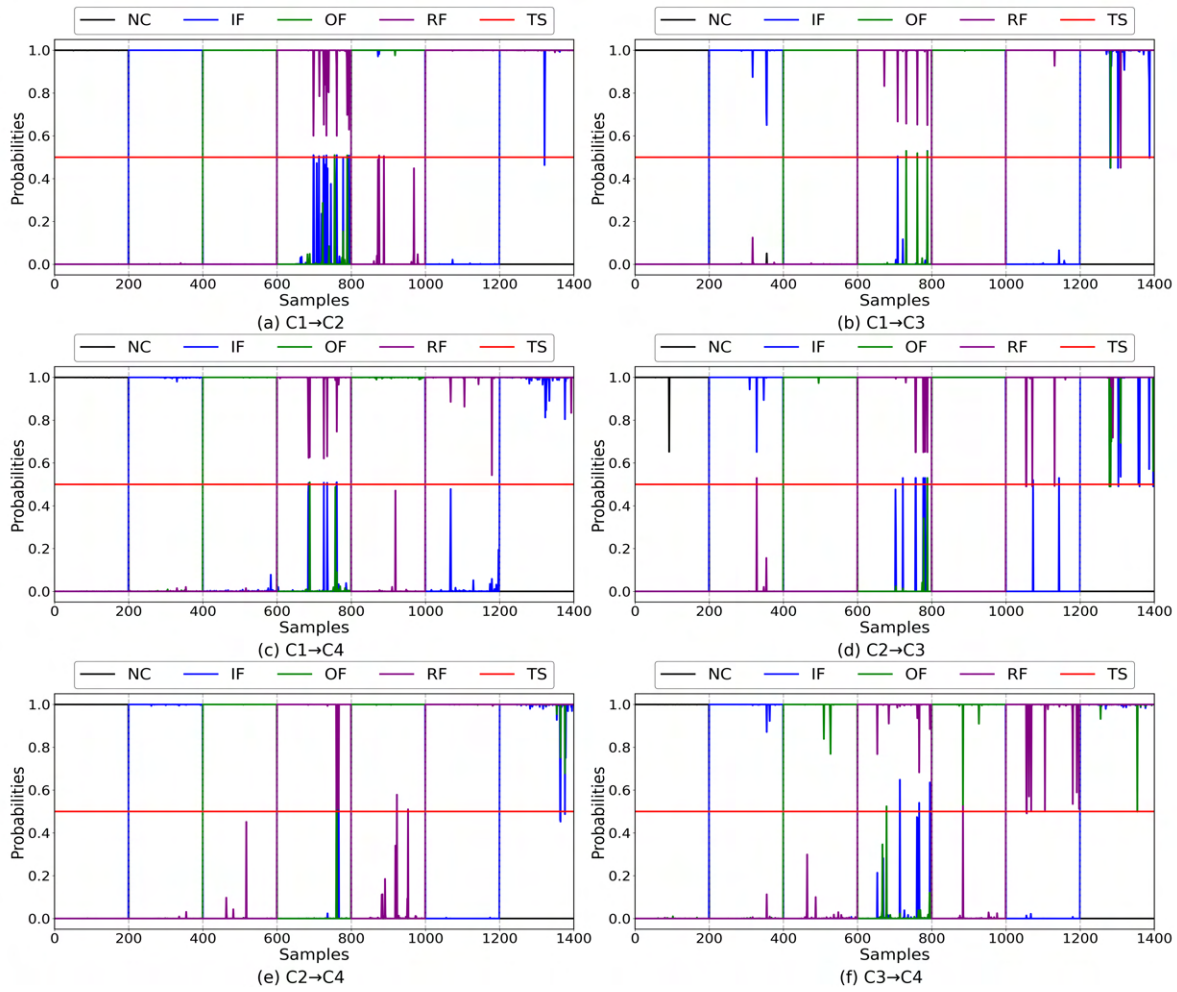


Figure 5: Probabilities fitting each fault for all samples on tasks C1→C2 (a), C1→C3 (b), C1→C4 (c), C2→C3 (d), C2→C4 (e) and C3→C4 (f) on BJTU-RAO dataset, TS represents the threshold value h .

For most compound fault samples, the probabilities of the relevant fault classes can stably remain above the threshold. Only a few compound fault samples are misclassified, in these misclassified samples, the probability of a single fault class within the compound

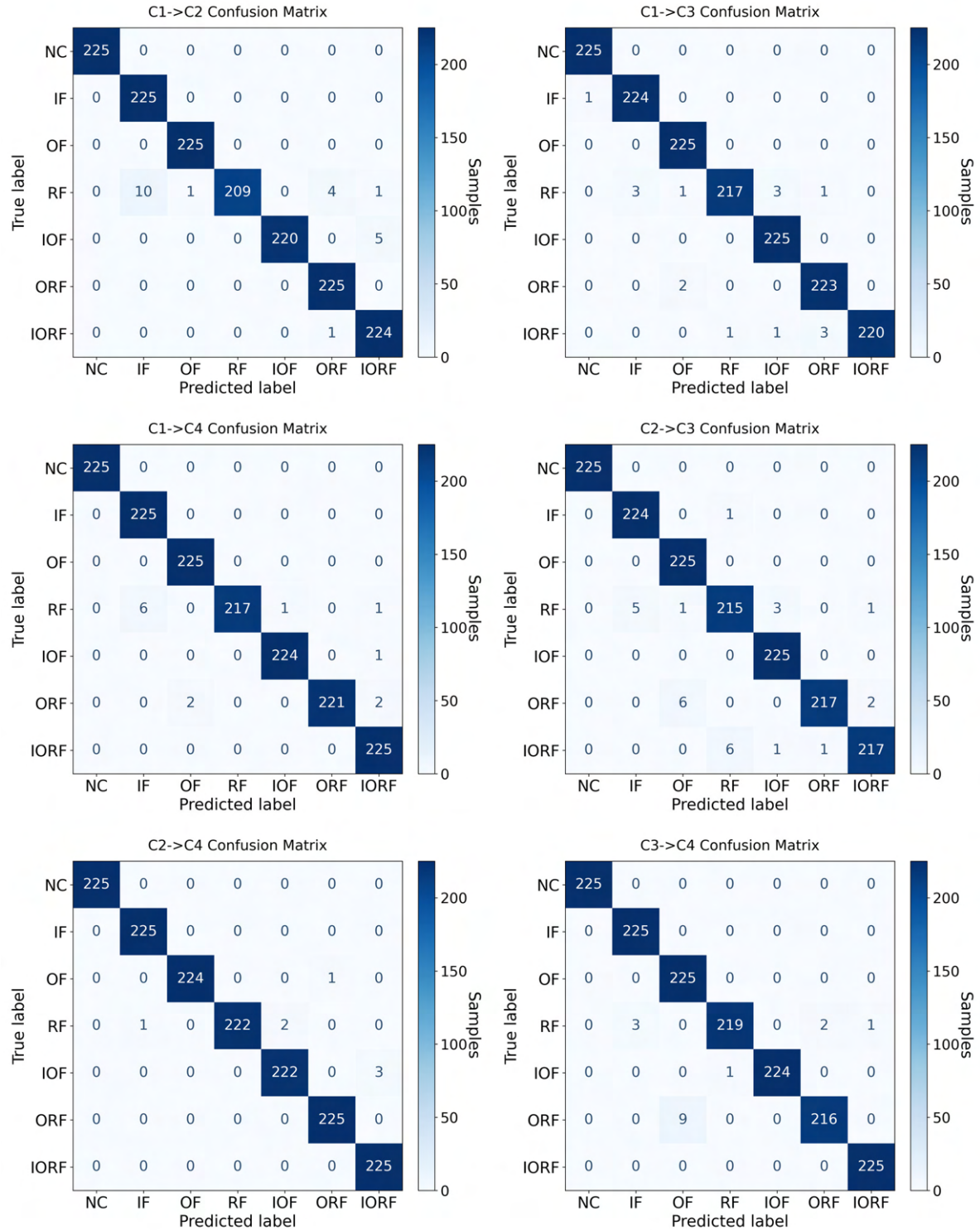


Figure 6: Confusion matrices of different tasks on BJTU-RAO dataset.

fault is either higher or lower than the threshold.

Figure 6 shows the confusion matrices of Dashnosis on six transfer diagnosis tasks. The results are mainly concentrated on the diagonal, indicating that most normal, single-

fault, and compound-fault samples are correctly classified. Only a few misclassifications occur between fault types with similar vibration characteristics, especially RF, ORF, and IORF. This is reasonable because these faults may contain overlapping impulse responses and coupled frequency components. Overall, the confusion matrices further demonstrate the stable classification performance and interpretability of Dashnosis under cross-operating-condition compound fault diagnosis.

4.4. Test results with different parameters

To evaluate the performance of the Dashnosis method under different parameter configurations, we varied the number of encoder layers, the number of decoder layers, the value of the domain loss parameter and the number of split patches. Corresponding fault diagnosis experiments were then conducted for each configuration to assess their impact on method performance.

To estimate the impact of the number of encoder layers on the diagnostic performance, we conducted experiments by increasing the number of encoder layers from 1 to 4. As shown in Figure 7(a), the performance first increases and then decreases as the number of encoder layers increases. The best performance is obtained when the number of encoder layers is 3. This result indicates that a moderate encoder depth can improve the extraction of discriminative time-frequency features and domain-invariant representations. However, an overly deep encoder may introduce redundant feature propagation or weaken local fault-related patterns, especially under limited training samples. Therefore, three encoder layers are selected as an empirical optimum under the current experimental setting, providing a suitable balance between feature representation and generalization performance.

To evaluate the impact of the number of decoder layers on the diagnostic performance, we conducted experiments by increasing the number of decoder layers from 1 to 4. As shown in Figure 7(b), the performance gradually improves when the number of decoder layers increases from 1 to 3, indicating that a moderate decoder depth can enhance class-relevant feature selection and suppress redundant or interfering features. However, when the number of decoder layers is further increased to 4, the performance declines. This may be because excessive layer-wise filtering weakens some class-related features and reduces class separability. Therefore, three decoder layers are selected as an empirical setting that balances feature filtering ability and classification performance.

To study the impact of the value of the domain loss parameter on method performance, we conducted a set of experiments. As shown in Figure 7(c), when the value increased from 0.3 to 0.9, the performance of the method improved first and then decreased. The method performed best when the value was 0.5. This indicates that moderately increasing the domain loss helps improve the cross-domain capability of method, allowing the encoder to extract sufficient domain-invariant features, thereby enhancing the accuracy of fault diagnosis. However, as the domain loss parameter

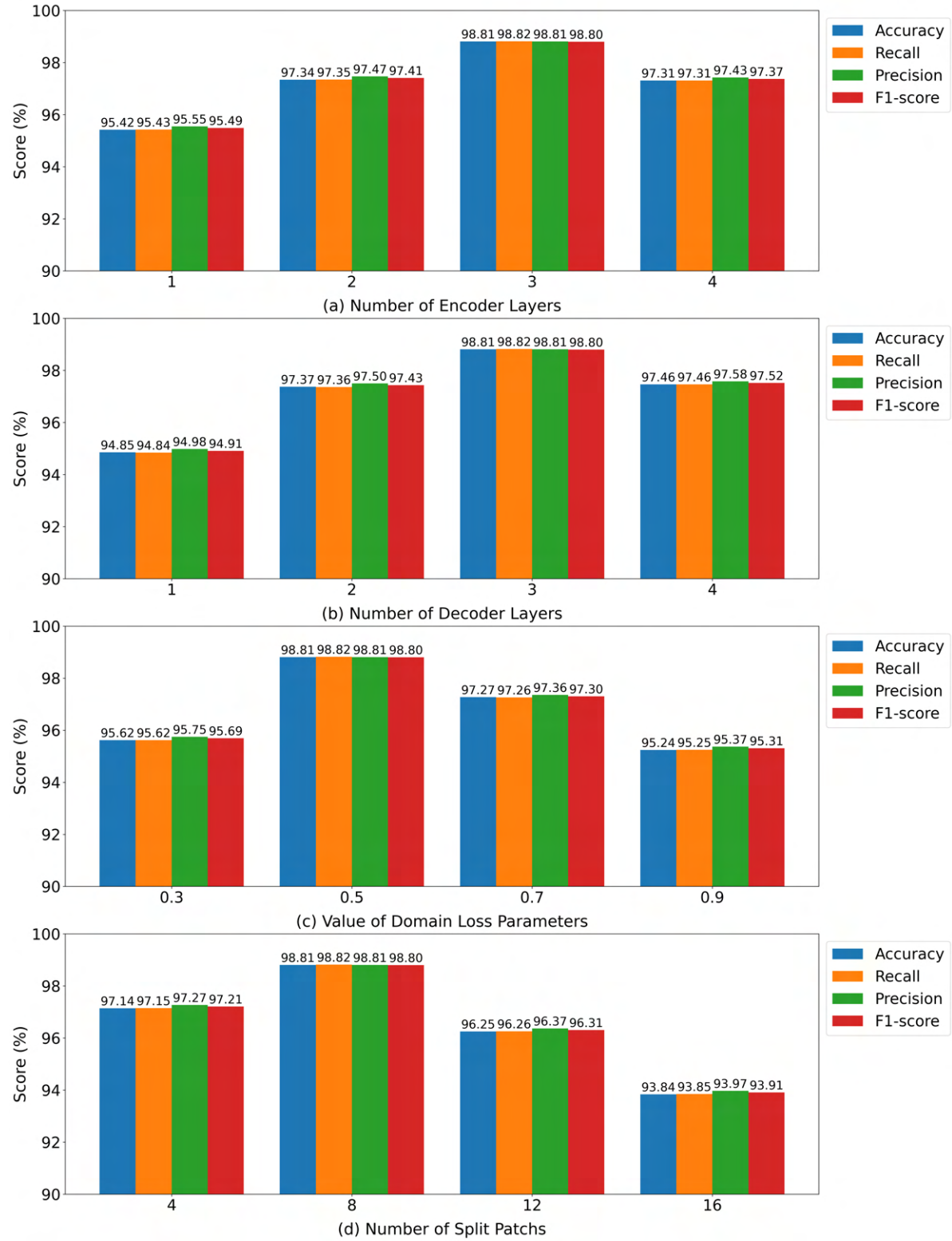


Figure 7: Performance bar chart of different parameters on BJTU-RAO dataset.

continued to increase to 0.7 and 0.9, the method performance gradually declined. This may occur because when the domain loss accounts for too much in the total loss

function, it suppresses the dominant role of the classification loss, ultimately leading to a decrease in overall method performance. Therefore, according to the experimental results, setting the domain loss parameter to 0.5 can achieve better performance. Note that this value is selected based on the tested candidate settings, rather than being mathematically proven as the global optimum. Since continuous hyperparameter values cannot be exhaustively evaluated, 0.5 is regarded as an empirical setting that achieves favorable performance under the current experimental configuration. More fine-grained hyperparameter optimization, such as dense grid search, Bayesian optimization, or adaptive weight adjustment, will be considered in future work.

In order to analyze the impact of the number of split patches on method performance, while keeping the number of encoder layers, decoder layers and domain loss parameters unchanged, we divided the input sequences into 4, 8, 12 and 16 patches for comparison experiments. As shown in Figure 7(d), when the number of patches increased from 4 to 8, the performance of the method improved significantly, reaching the best performance with 8 patches. This indicates that appropriately segmenting the input sequence helps enhance the ability to extract local features, allowing the method to fully capture fine-grained features while retaining global features, thereby improving the accuracy of fault diagnosis. However, when the number of patches further increased to 12 and 16, method performance gradually declined. This may be because too many patches result in very small individual patch sizes, with each patch containing fewer features, which affects the feature extraction capability. The experimental results indicate that dividing the input into 8 patches can achieve the optimal performance of method.

4.5. Test results for component importance

To examine the effect of individual Dashnosis method components on diagnostic performance, we implement an ablation study that systematically omits key constituents and conducts fault diagnosis experiments. The core components of the Dashnosis method include the feature transfer enhancement encoder, the multi-class feature filtering encoder, the progressive feature transfer approach, the irrelevant feature filtering approach and the time-frequency feature enhancement approach. First, we remove the multi-class feature filtering decoder while retaining the feature transfer enhancement encoder to obtain a variant referred to as Dashnosis-I, which is then evaluated. Based on the original method, we further remove the feature transfer enhancement encoder while retaining the multi-class feature filtering decoder to obtain Dashnosis-II. Next, we test three additional variants derived from the original Dashnosis: Dashnosis-III, which removes only the progressive feature transfer approach; Dashnosis-IV, which removes the irrelevant feature filtering approach and replaces it with a single-layer CNN; and Dashnosis-V, which removes only the time-frequency feature enhancement approach. As shown in Table 5, directly removing the encoder or decoder significantly reduces the performance of method. In addition, similar performance drops were observed in experiments with three additional variants. These results indicate

Table 5: Ablation study results on BJTU-RAO dataset (%).

Method	Accuracy	Recall	Precision	F1-score
Dashnosis-I	77.08%	77.08%	77.15%	77.11%
Dashnosis-II	87.56%	87.59%	87.62%	87.60%
Dashnosis-III	90.35%	90.36%	90.47%	90.41%
Dashnosis-IV	95.24%	95.17%	95.21%	95.18%
Dashnosis-V	96.87%	96.85%	96.96%	96.90%
Dashnosis	98.81%	98.82%	98.81%	98.80%

Table 6: Signal-to-noise ratio test results on BJTU-RAO dataset (%).

Task	-10dB	-5dB	0dB	5dB	10dB	Average
C1→C2	69.21%	90.03%	95.56%	97.33%	98.22%	90.07%
C1→C3	67.37%	84.89%	95.68%	97.08%	98.02%	88.41%
C1→C4	58.52%	82.62%	94.86%	97.52%	98.41%	86.39%
C2→C3	62.54%	85.41%	95.24%	96.89%	97.27%	87.47%
C2→C4	66.35%	85.02%	95.56%	98.22%	98.86%	88.80%
C3→C4	70.29%	89.46%	96.13%	98.10%	98.54%	90.50%

that each component contributes positively to the overall diagnostic performance of the method.

4.6. Test results of noise robustness

To evaluate the robustness of Dashnosis against noise interference, we further conducted noise robustness experiments on BJTU-RAO dataset. In this experiment, Gaussian white noise with different signal-to-noise ratio (SNR) levels was added to the testing signals, while the training settings remained unchanged. The SNR levels were set to -10dB, -5dB, 0dB, 5dB and 10dB. As shown in Table 6, Dashnosis maintains high diagnostic accuracy under moderate noise conditions. When the SNR is 10dB and 5dB, the average accuracies over the six transfer tasks reach 98.22% and 97.52%, respectively. Even under 0dB noise, the average accuracy remains 95.51%, indicating that the model can effectively resist moderate noise interference. When the noise becomes severe, the performance gradually decreases, with the average accuracy dropping to 86.24% at -5dB and 65.71% at -10dB. This degradation is reasonable because strong noise can obscure fault related impulse components and frequency-domain energy distributions. Overall, the results demonstrate that Dashnosis has good noise robustness and can maintain reliable diagnostic performance under noisy testing conditions.

Table 7: Characteristics of compared methods and parameter settings.

Method	Zero-shot learning	Domain adaptation	Parameters
DDCNN	Yes	No	See Table 3 in [27]
WavCapsNet	Yes	No	See Fig.4 in [28]
TCN	Yes	Yes, single domain	See Fig.5 in [29]
DACNs	Yes	Yes, single domain	See Fig.11 in [30]
DACNm	Yes	Yes, multi-domains	See Fig.11 in [30]

Table 8: Diagnosis accuracies of various methods on BJTU-RAO dataset (%).

Method	C1→C2	C1→C3	C1→C4	C2→C3	C2→C4	C3→C4	Average
DDCNN	84.06%	84.70%	85.78%	85.65%	85.40%	83.68%	84.88%
WavCapsNet	85.21%	84.76%	86.09%	86.15%	86.67%	85.84%	85.77%
TCN	89.21%	88.76%	91.11%	90.34%	87.87%	89.97%	89.54%
DACNs	92.25%	91.75%	93.71%	93.27%	92.19%	93.02%	92.70%
DACNm	95.17%	92.44%	96.12%	95.62%	94.67%	95.24%	94.88%
Dashnosis	98.35%	98.98%	99.30%	98.10%	99.37%	98.73%	98.81%

4.7. Test results compared with existing methods

Given that only a limited number of studies simultaneously consider zero-shot compound fault diagnosis and domain adaptation, Dashnosis is evaluated against several representative methods, including DDCNN [27], WavCapsNet [28], TCN [29] and DACN [30]. These approaches share a common characteristic in that they can be trained using only single-fault samples and subsequently applied to compound fault diagnosis. The primary difference among them lies in the strategies used for domain adaptation. DDCNN and WavCapsNet do not incorporate any adaptation mechanism. TCN performs conventional single-domain adaptation from the source domain to the target domain. DACN is further divided into two variants, DACNs and DACNm. DACNs follows the same source-to-target adaptation paradigm as TCN, whereas DACNm conducts multi-domain alignment among several source domains without explicitly adapting to the target domain. For instance, in the C1→C2 task, DACNm aligns the feature distributions of datasets C1, C3 and C4, while DACNs directly transfers knowledge from C1 to C2.

A detailed comparison of method characteristics and parameter settings is provided in Table 7. The diagnostic accuracy results are summarized in Table 8 and Table A2 shows the corresponding metrics such as recall, precision, and F1 score. Figure 8 demonstrates that Dashnosis consistently achieves the best performance across all evaluated tasks. Methods without domain adaptation, namely DDCNN and WavCapsNet, show the lowest accuracy. Although both TCN and DACNs rely on

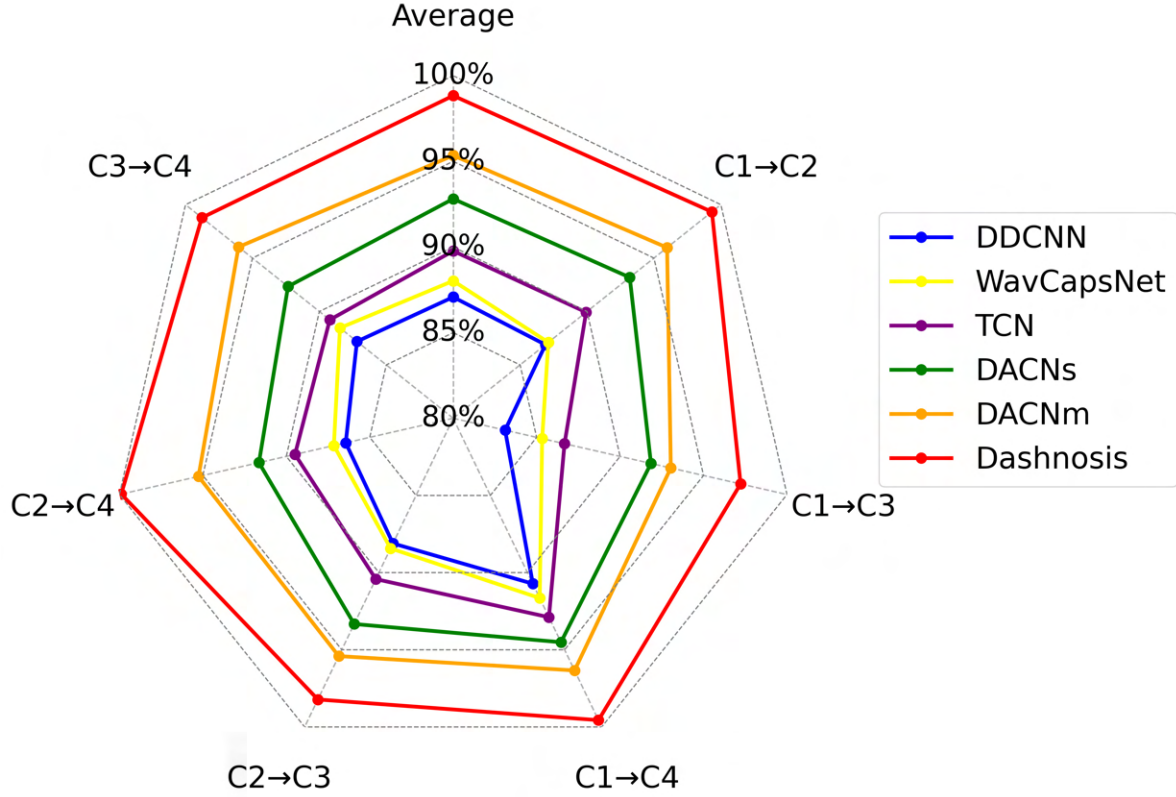


Figure 8: Radar diagram of Dashnosis accuracies on BJTU-RAO dataset (%).

single-domain adaptation, DACNs achieves better results due to its more advanced architecture. By leveraging multi-source domain alignment, DACNm further improves the diagnostic performance over the other baselines. Nevertheless, Dashnosis still surpasses DACNm despite using only single-domain adaptation.

To further interpret the performance differences, the t-SNE visualization of feature distributions for the C2→C4 task is presented in Figure 9. In the source domain, most methods can separate different fault categories to some extent, while WavCapsNet, DACNs, DACNm and Dashnosis exhibit more distinct class boundaries compared with DDCNN and TCN. From the perspective of cross-domain alignment, TCN, DACNs, DACNm and Dashnosis produce noticeably more compact clusters for samples belonging to the same fault category across domains, whereas DDCNN and WavCapsNet show more dispersed distributions due to the absence of domain adaptation mechanisms. Furthermore, in the target domain, Dashnosis forms the most clearly separated clusters among all the methods compared.

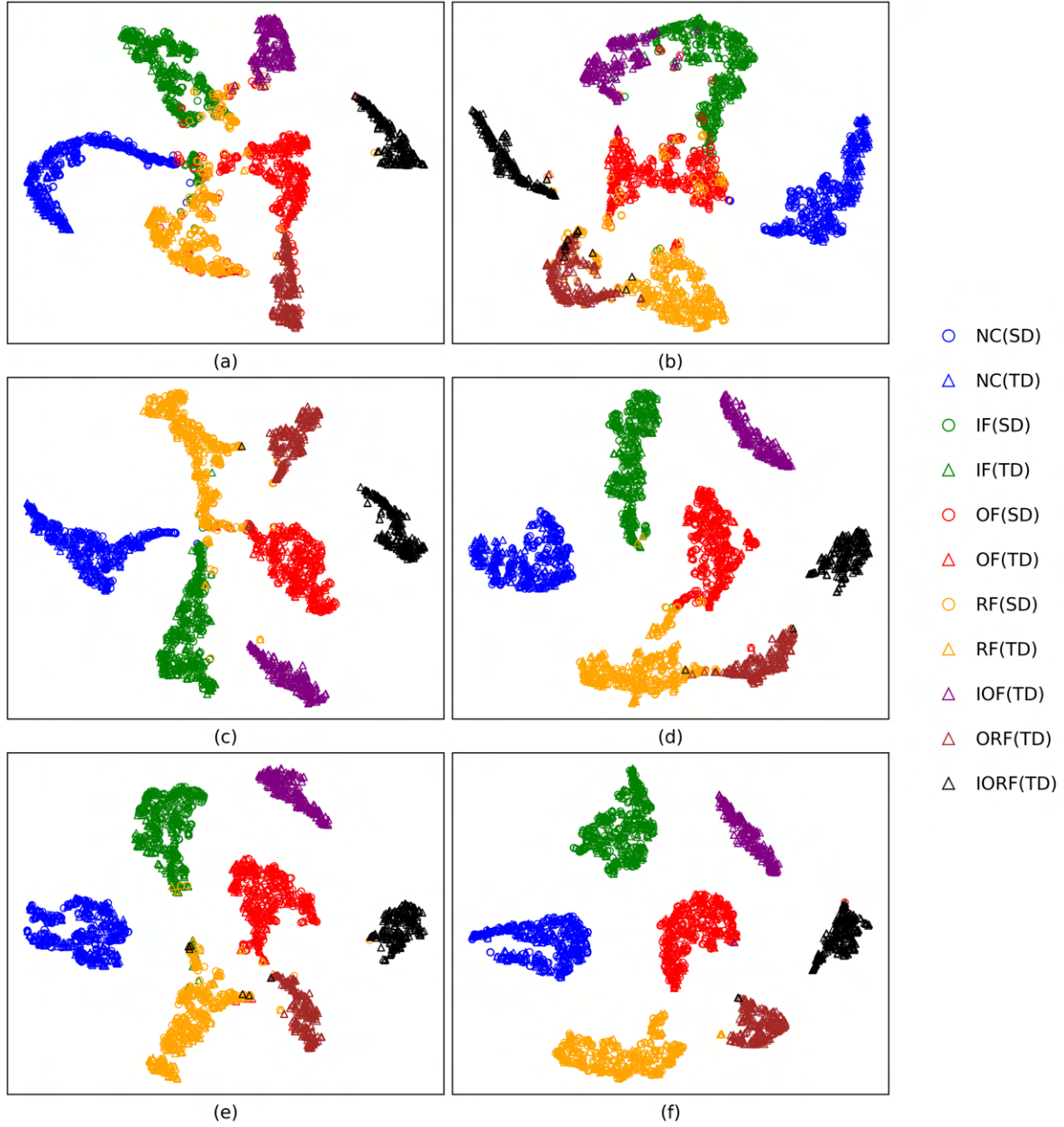


Figure 9: Visualization of feature distributions on task C2→C4 on BJTU-RAO dataset. (a) DDCNN. (b) WavCapsNet. (c) TCN. (d) DACNs. (e) DACNm. (f) Dashnosis.

5. Case study II: performance evaluation based on HUSTbearing dataset

5.1. The introduction of the HUSTbearing dataset

To further evaluate the cross operating condition compound fault diagnosis performance of the proposed Dashnosis model, the HUSTbearing dataset is used in this study [31]. The dataset was collected on a Spectra-Quest Mechanical Fault Simulator, which consists of a speed controller, motor, shaft, acceleration sensor, test bearing, and data acquisition board. The tested bearing type is ER-16K. In this study, vibration signals

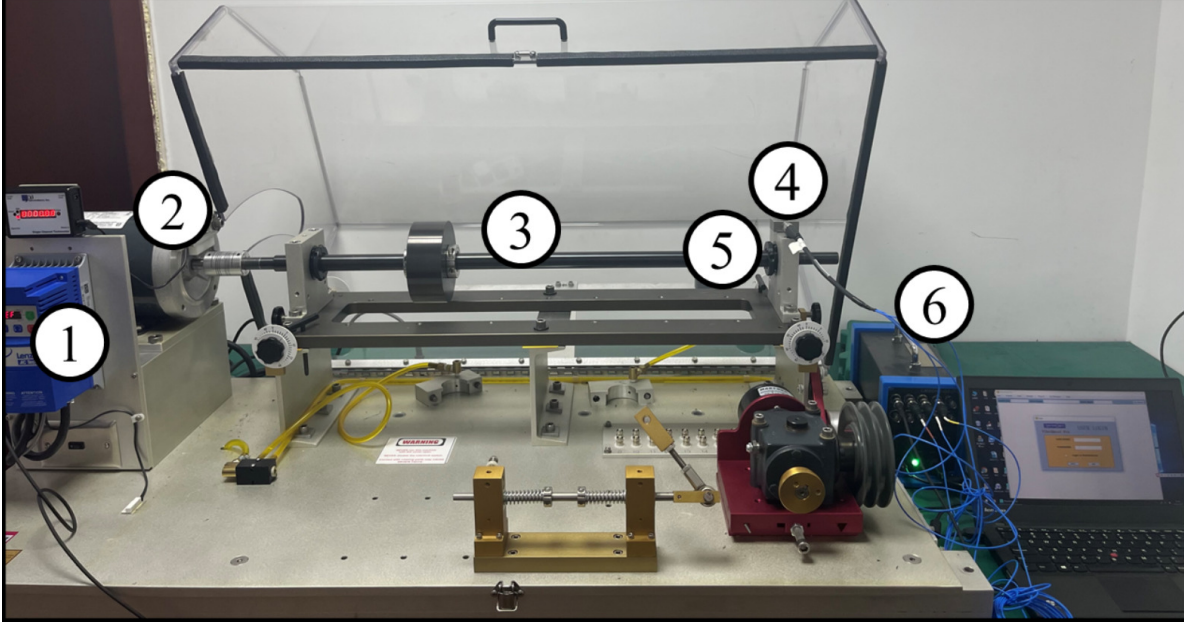


Figure 10: HUSTbearing dataset experimental platform.

Table 9: Details of the HUSTbearing dataset for training and testing.

Dataset	Working condition	Fault type for training	Fault type for testing	Samples
C1	20Hz	NC, MI, MO, SI, SO	—	800
C2	40Hz	NC, MI, MO, SI, SO	NC, MI, MO, SI, SO, MC, SC	1400
C3	60Hz	NC, MI, MO, SI, SO	NC, MI, MO, SI, SO, MC, SC	1400
C4	80Hz	NC, MI, MO, SI, SO	NC, MI, MO, SI, SO, MC, SC	1400

under four constant-speed operating conditions are selected, namely 20 Hz, 40 Hz, 60 Hz and 80 Hz, which are denoted as C1, C2, C3 and C4, respectively. Seven health states are considered, including normal condition (NC), medium inner-race fault (MI), medium outer-race fault (MO), severe inner-race fault (SI), severe outer-race fault (SO), medium combination fault (MC), and severe combination fault (SC). The combination fault refers to the simultaneous fault of the inner race and outer race. An image of the experimental platform is shown in Figure 10. The sampling frequency is 25.6 kHz, and 262144 data points are recorded for each sampling. In each transfer task, NC, MI, MO, SI and SO are used for training, while NC, MI, MO, SI, SO, MC and SC are used for testing, as summarized in Table 9. Based on the four operating conditions, six transfer diagnosis tasks are constructed, including C1→C2, C1→C3, C1→C4, C2→C3, C2→C4 and C3→C4. Figure 11 illustrates the signals of different fault types in dataset C2.

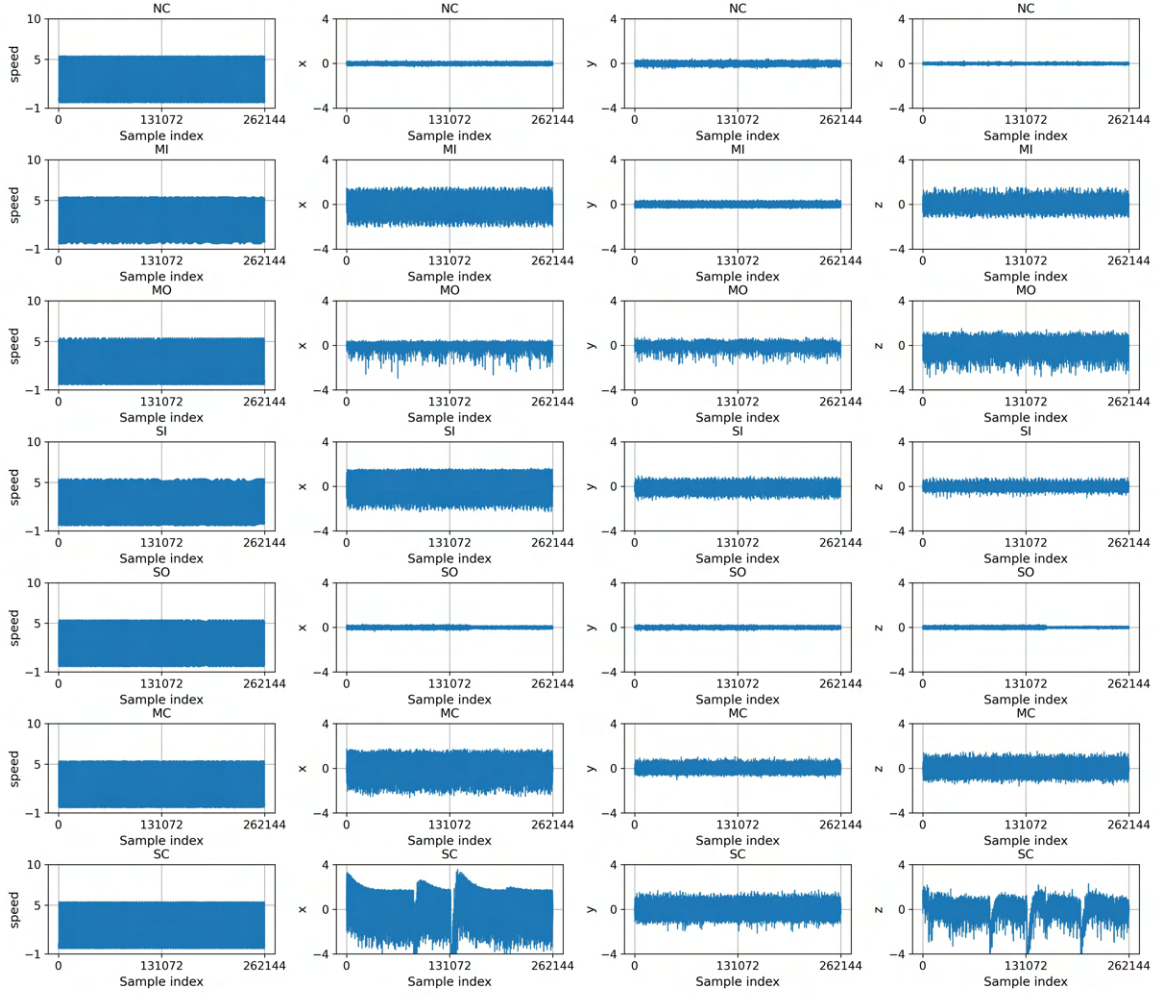


Figure 11: Signals of each fault in dataset C2 on HUSTbearing dataset.

5.2. Test results on different tasks

Table 10 shows the diagnosis accuracies of Dashnosis on six transfer tasks using the HUSTbearing dataset, while Table A3 shows the corresponding metrics such as recall, precision, and F1 score. It can be observed that Dashnosis achieves stable performance under different source-target domain combinations. The average accuracies of all tasks are higher than 97%, and the overall average accuracy reaches 98.46%. Among all transfer tasks, C1→C2 obtains the highest average accuracy of 98.79%, while C2→C4 still reaches 97.59%, demonstrating that Dashnosis can maintain reliable diagnosis performance when the operating condition changes. For different fault classes, both single fault classes and compound fault classes can be accurately identified. These results indicate that the proposed method also has good generalization ability on HUSTbearing dataset.

Figure 12 further illustrates the predicted probability distributions of all test samples. Most samples show high predicted probabilities for their corresponding fault

Table 10: Dashnosis accuracies of each fault on HUSTbearing dataset (%).

Task	NC	MI	MO	SI	SO	MC	SC	Average
C1→C2	100.00%	96.44%	100.00%	100.00%	95.56%	100.00%	99.56%	98.79%
C1→C3	100.00%	98.67%	99.56%	100.00%	96.89%	98.67%	97.33%	98.73%
C1→C4	100.00%	100.00%	100.00%	100.00%	99.56%	98.22%	92.44%	98.73%
C2→C3	100.00%	96.44%	99.56%	100.00%	92.00%	100.00%	100.00%	98.29%
C2→C4	94.67%	100.00%	100.00%	95.11%	97.78%	100.00%	95.56%	97.59%
C3→C4	95.11%	100.00%	100.00%	100.00%	98.22%	97.78%	100.00%	98.73%

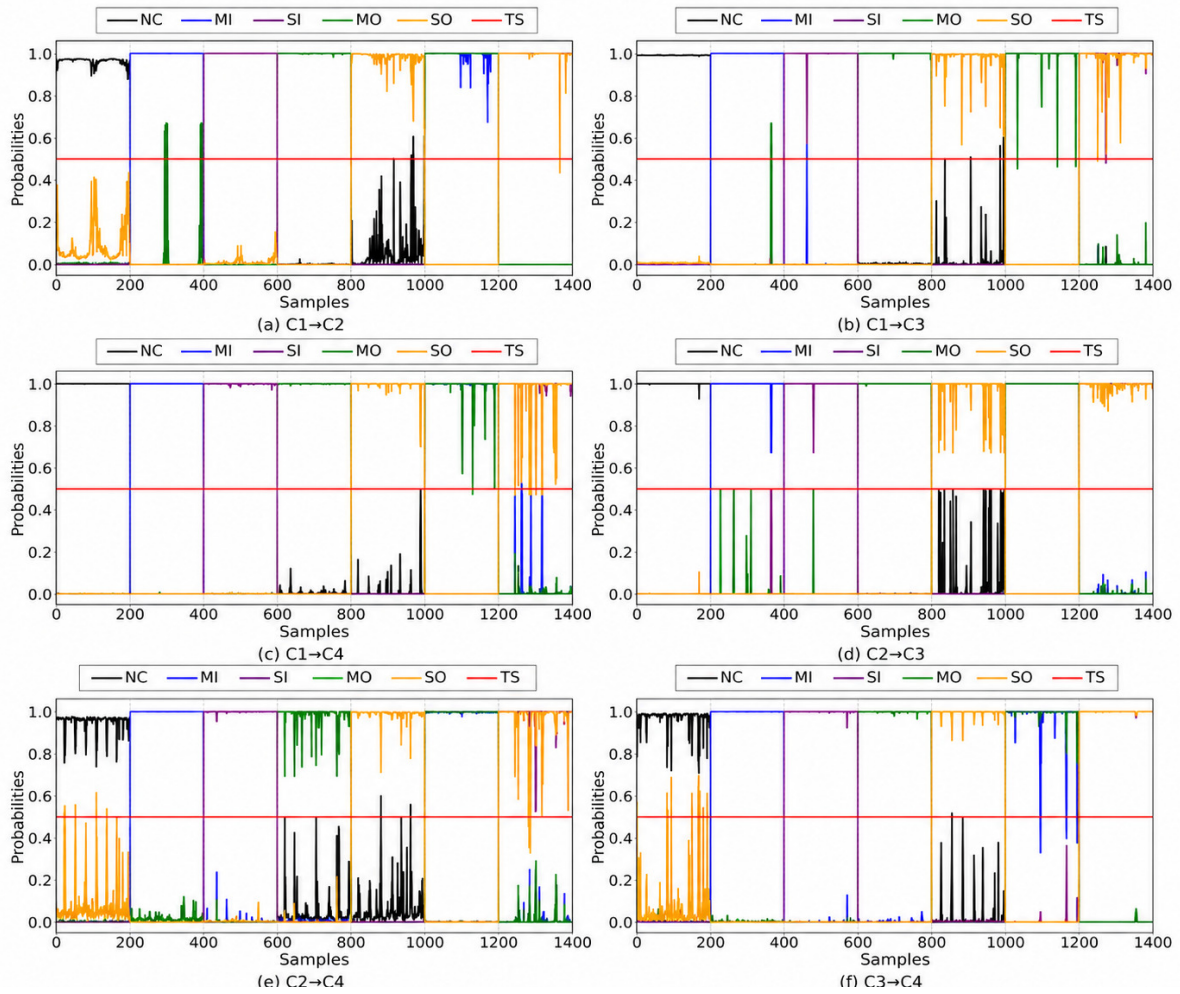


Figure 12: Probabilities fitting each fault for all samples on tasks C1→C2 (a), C1→C3 (b), C1→C4 (c), C2→C3 (d), C2→C4 (e) and C3→C4 (f) on HUSTbearing dataset, TS represents the threshold value h .

classes, and the probability responses of different classes are clearly distinguishable. For compound fault samples, the related single-fault responses are also effectively activated, indicating that Dashnosis can identify multiple coexisting fault components from the

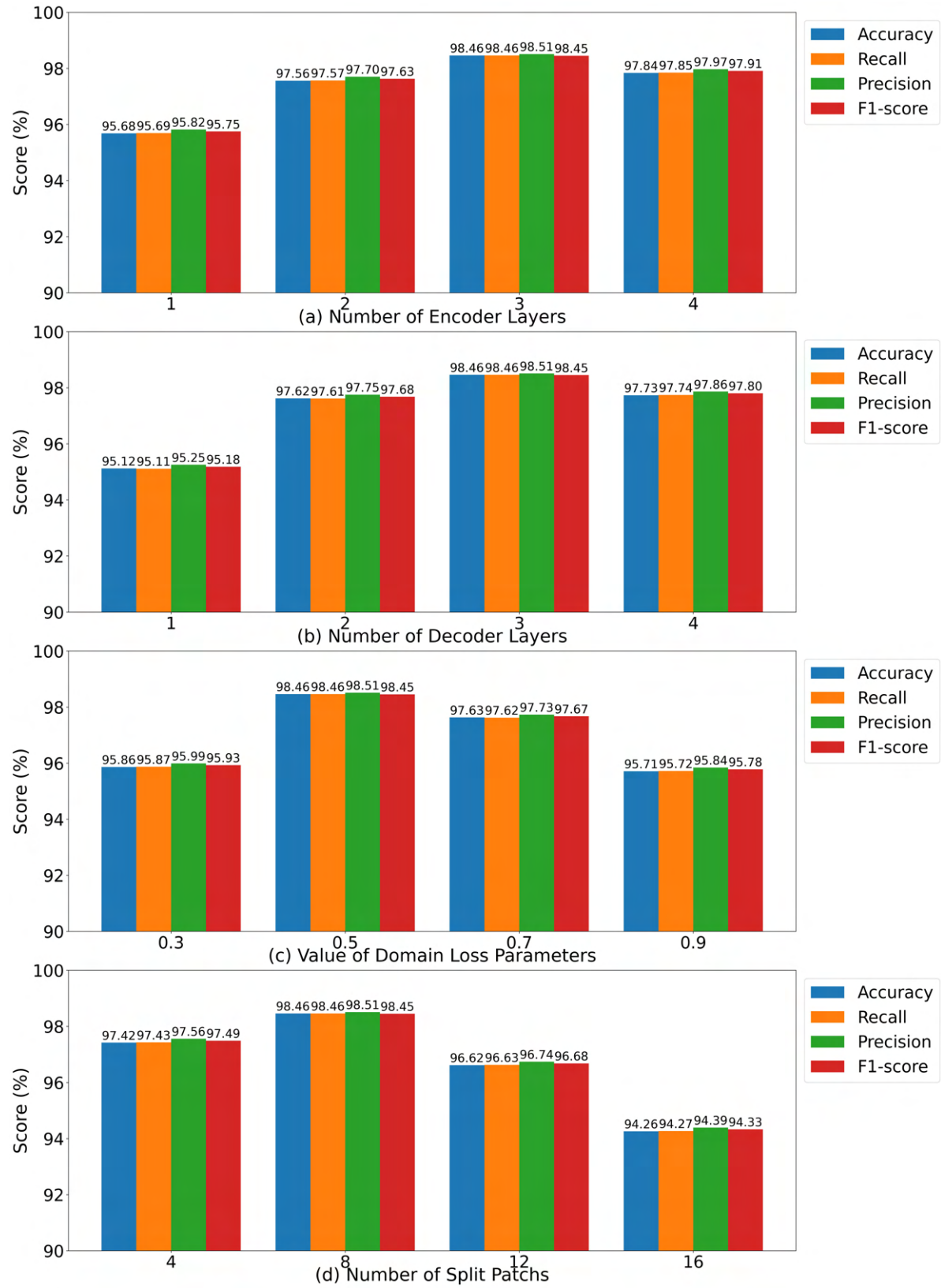


Figure 13: Performance bar chart of different parameters on HUSTbearing dataset.

Table 11: Ablation study results on HUSTbearing dataset (%).

Method	Accuracy	Recall	Precision	F1-score
Dashnosis-I	76.42%	76.31%	76.50%	76.40%
Dashnosis-II	86.91%	86.78%	86.94%	86.86%
Dashnosis-III	89.67%	89.74%	89.82%	89.78%
Dashnosis-IV	94.48%	94.53%	94.61%	94.57%
Dashnosis-V	96.12%	96.03%	96.24%	96.13%
Dashnosis	98.46%	98.47%	98.44%	98.45%

coupled fault representation. Only a small number of samples show relatively weak or fluctuating responses, which may be caused by similar vibration characteristics between fault classes with close fault severity. Overall, the probability distribution results further verify the effectiveness of Dashnosis in cross operating condition compound fault diagnosis on HUSTbearing dataset.

5.3. Test results with different parameters

Figure 13(a)-(d) shows the influence of different parameters on the diagnosis performance of Dashnosis. The overall variation trend is consistent with that observed on BJTU-RAO dataset. Appropriate parameter settings can improve the feature extraction ability and cross-domain ability of the method, thereby enhancing the diagnosis performance under different operating conditions. However, excessively increasing model depth or feature filtering strength does not always bring further improvement. It may introduce redundant feature propagation or weaken some key fault-related features during layer-wise transmission. Therefore, the parameter experimental results on HUSTbearing dataset further demonstrate that the selected parameter settings provide a suitable balance between feature representation capability and generalization performance.

5.4. Test results for component importance

Table 11 presents the ablation results of Dashnosis on HUSTbearing dataset. The complete Dashnosis model achieves the best results among all variants, with an accuracy of 98.46%, recall of 98.47%, precision of 98.44%, and F1-score of 98.45%. When different key components are removed, the diagnosis performance decreases to different degrees, indicating that each component contributes to the final performance. In particular, removing the multi-class feature filtering decoder causes the most significant performance degradation, which demonstrates that class relevant feature filtering is important for identifying compound fault components when compound fault samples are unavailable during training. Removing the feature transfer enhancement encoder also leads to an obvious decline, verifying its role in extracting domain invariant features

Table 12: Diagnosis accuracies of various methods on HUSTbearing dataset (%).

Method	C1→C2	C1→C3	C1→C4	C2→C3	C2→C4	C3→C4	Average
DDCNN	83.81%	84.55%	85.52%	85.90%	85.14%	83.94%	84.86%
WavCapsNet	85.46%	84.71%	86.35%	86.31%	86.92%	85.59%	85.89%
TCN	88.31%	88.46%	90.35%	90.22%	87.25%	89.64%	89.04%
DACNs	91.86%	90.95%	93.43%	92.88%	92.03%	92.37%	92.25%
DACNm	94.55%	92.02%	95.84%	95.18%	94.01%	94.63%	94.37%
Dashnosis	98.79%	98.73%	98.60%	98.29%	97.59%	98.73%	98.46%

and reducing the distribution discrepancy between the source and the target domain.

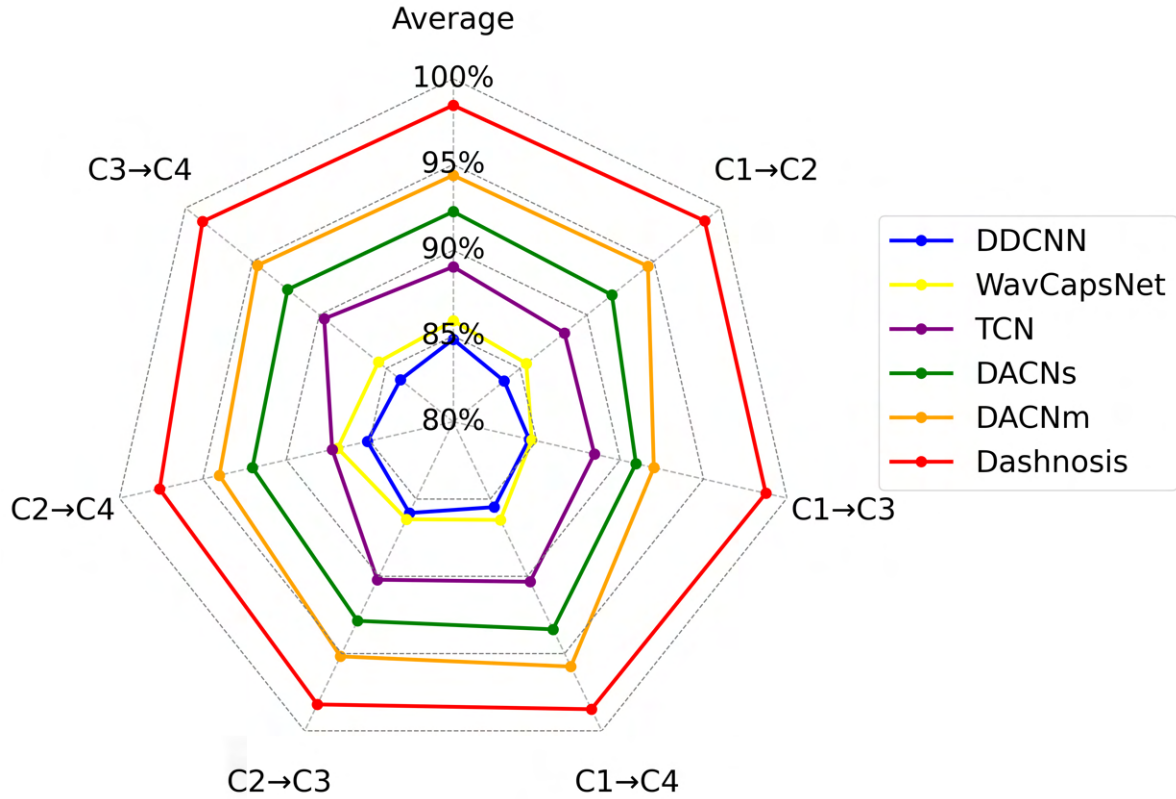


Figure 14: Radar diagram of Dashnosis accuracies on HUSTbearing dataset (%).

5.5. Test results compared with existing methods

Table 12 shows the comparison results of different methods on HUSTbearing dataset, while Table A4 shows the corresponding metrics such as recall, precision, and F1 score. Dashnosis achieves the highest accuracy on all six transfer tasks, with an average accuracy of 98.46%. Compared with DACNm, which obtains the second-best average accuracy of 94.37%, Dashnosis improves the average performance by 4.09%. The radar

diagram in Figure 14 also shows that Dashnosis maintains consistently high accuracy under different transfer settings, while the compared methods show relatively larger performance fluctuations.

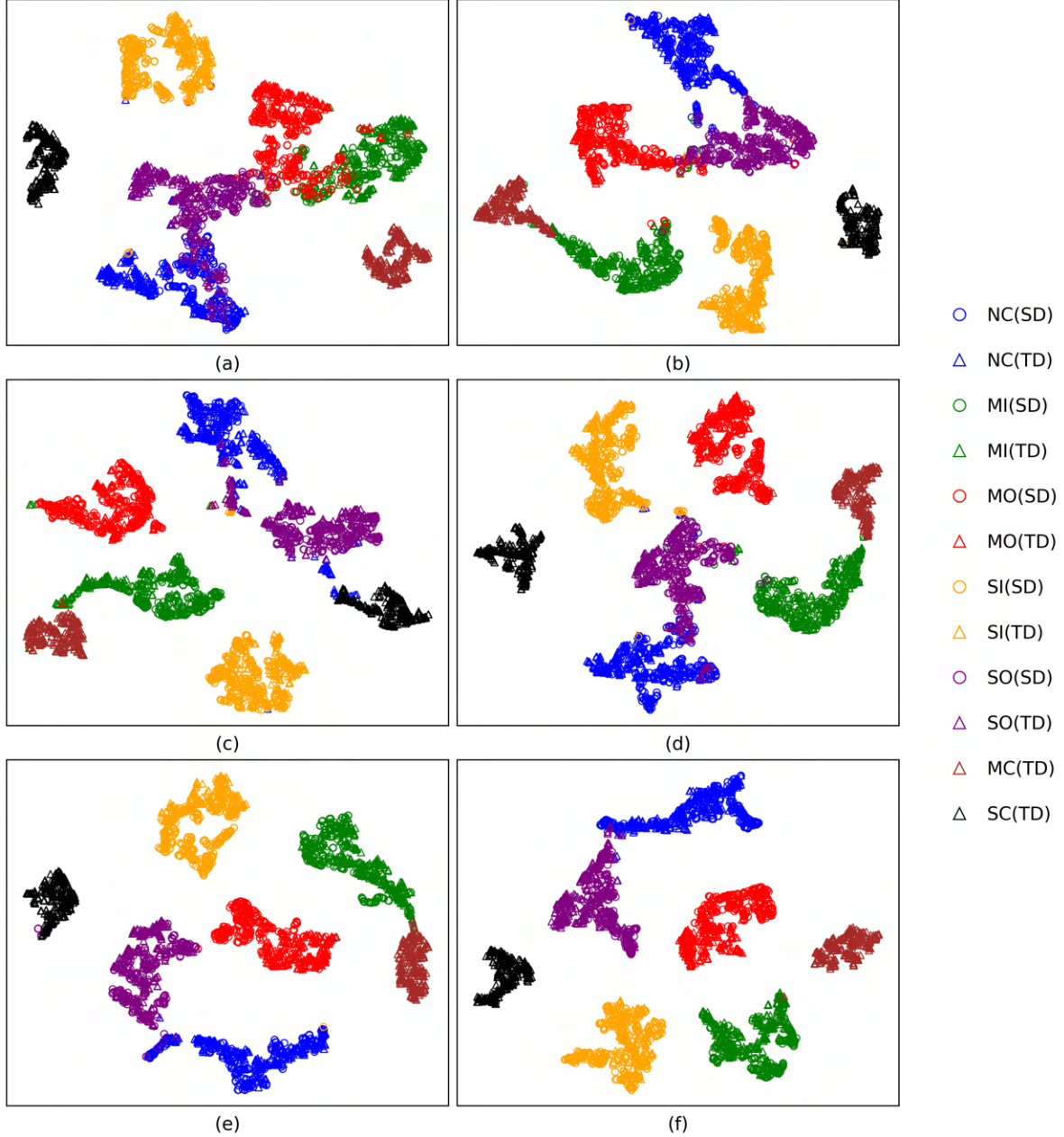


Figure 15: Visualization of feature distributions on task C3→C4 on HUSTbearing dataset. (a) DDCNN. (b) WavCapsNet. (c) TCN. (d) DACNs. (e) DACNm. (f) Dashnosis.

Further feature visualization is shown in Figure 15. For DDCNN, WavCapsNet and TCN, the source-domain and target-domain features are not sufficiently aligned, and some fault classes still show obvious overlap. DACNs and DACNm improve the feature distribution to some extent, but the class boundaries remain less compact.

In contrast, Dashnosis produces more compact intra-class distributions and clearer inter-class separation. The source-domain and target-domain samples of the same fault class are closer in the feature space, indicating that Dashnosis can better extract domain invariant features and class relevant features. These results further verify the effectiveness of Dashnosis for cross operating condition compound fault diagnosis on HUSTbearing dataset.

6. Conclusions

We propose Dashnosis that can be trained using only single class fault data from the source domain and accurately diagnose compound faults in the target domain, thereby solving the issues of scarcity of compound fault data and domain shift. In Dashnosis, we propose the multi-class feature filtering decoder that combines the CSRA framework and the FAB framework. In this decoder, we design irrelevant feature filtering approach and branch freezing training approach achieving compound fault diagnosis under the condition of zero compound fault samples. Meanwhile, we propose the feature transfer enhancement encoder based on the TVT framework. We design progressive feature transfer approach and time-frequency feature enhancement approach which effectively alleviate the domain shift and improve the encoder cross-domain capability. Extensive experiments on six tasks using the BJTU-RAO dataset and HUSTbearing dataset demonstrate that Dashnosis can effectively distinguish features of different compound faults and achieve robust feature clustering and transfer across varying operating conditions, thereby enabling accurate compound fault diagnosis. In future work, we plan to incorporate tensor-based domain-adversarial adaptation [32], feature mode decomposition [33] and multi-modal feature fusion [34] and related techniques to further extend the applicability of compound fault diagnosis across heterogeneous machines.

Declarations

Competing interests

The authors declare that there is no conflict of interest.

Ethical and informed consent for data used

This study does not contain any studies with human or animal subjects performed by any of the authors.

Data availability and access

BJTU-RAO bearing dataset: <https://drive.google.com/drive/folders/1R1ZvFw-v07VvsL2Ni9cS7iFrTPDIhn2r?usp=sharing>

HUSTbearing dataset: https://drive.google.com/drive/folders/1UM0vyfstYJRyR0rPw00fH-tIjJg2_0aN?usp=sharing

Acknowledgement

We acknowledge financial support from CRRC Academy.

References

- [1] Wanchun Hu, Ge Xin, and Jiayi Wu. “Vibration-based bearing fault diagnosis of high-speed trains: A literature review”. In: *High-speed Railway* 1 (2023), pp. 219–223.
- [2] Yongjian Sun and Wei Wang. “Role of image feature enhancement in intelligent fault diagnosis for mechanical equipment: A review”. In: *Engineering Failure Analysis* 156 (2024), p. 107815.
- [3] Yiming Xiao, Haidong Shao, and Shen Yan. “Domain generalization for rotating machinery fault diagnosis: A survey”. In: *Advanced Engineering Informatics* 64 (2025), p. 103063.
- [4] Hao Chen, Jiaming Li, and Xianbo Wang. “Review of intelligent fault diagnosis for rotating machinery under imperfect data conditions”. In: *Expert Systems with Applications* 285 (2025), p. 127726.
- [5] Baosen Wang, Yongqiang Liu, and Qilan Li. “A review on research of system dynamics and multi-source fault diagnosis of key components in high-speed train”. In: *Chinese Journal of Mechanical Engineering* 39 (2026), p. 100066.
- [6] Juan Xu et al. “Zero-shot learning for compound fault diagnosis of bearings”. In: *Expert Systems with Applications* 190 (2022), p. 116197.
- [7] Juan Xu et al. “A label information vector generative zero-shot model for the diagnosis of compound faults”. In: *Expert Systems with Applications* 233 (2023), p. 120875.
- [8] Juan Xu et al. “CGASNet: A Generalized Zero-Shot Learning Compound Fault Diagnosis Approach for Bearings”. In: *IEEE Transactions on Instrumentation and Measurement* 73 (2024), pp. 1–11.
- [9] Juan Xu et al. “Generative Zero-Shot Compound Fault Diagnosis Based on Semantic Alignment”. In: *IEEE Transactions on Instrumentation and Measurement* 73 (2024), pp. 1–13.
- [10] Jian Cen et al. “An intelligent compound fault diagnosis method using generalized zero-shot model of bearing”. In: *Measurement Science and Technology* 35 (2024), p. 096134.
- [11] Lv Wang et al. “A zero-shot attribute-embedded model with a feature difference mapping sigmoid function for compound fault diagnosis of rotating machinery”. In: *ISA transactions* 157 (2025), pp. 451–465.

- [12] Shuangyan Yin et al. “A Zero-Shot Framework for Compound Fault Diagnosis in Rotating Machinery Using Orthogonal High-Order Semantics and Cross-Hierarchical Discriminative Learning”. In: *IEEE Transactions on Instrumentation and Measurement* 74 (2025), pp. 1–20.
- [13] Zhengming Xiao et al. “A hybrid semantic-based embedded zero-shot learning method for compound fault diagnosis of bearings”. In: *Measurement Science and Technology* 36 (2025), p. 116113.
- [14] Fengli Shen et al. “Zero-shot compound fault diagnosis with semantic graph embedding and multi-stage fusion”. In: *Neurocomputing* 668 (2026), p. 132389.
- [15] Chao Zhao and Weiming Shen. “Dual adversarial network for cross-domain open set fault diagnosis”. In: *Reliability Engineering and System Safety* 221 (2022), p. 108358.
- [16] Longzhang Ke, Yi Liu, and Yi Yang. “Compound Fault Diagnosis Method of Modular Multilevel Converter Based on Improved Capsule Network”. In: *IEEE Access* 10 (2022), pp. 41201–41214.
- [17] Xin Wang et al. “A dynamic collaborative adversarial domain adaptation network for unsupervised rotating machinery fault diagnosis”. In: *Reliability Engineering and System Safety* 255 (2024), p. 110662.
- [18] Yiming He, Chao Zhao, and Weiming Shen. “Cross-Domain Compound Fault Diagnosis of Machine-Level Motors via Time–Frequency Self-Contrastive Learning”. In: *IEEE Transactions on Industrial Informatics* 20 (2024), pp. 9692–9701.
- [19] Cailu Pan, Zhiwu Shang, and Lutai Tang. “Open-set domain adaptive fault diagnosis based on supervised contrastive learning and a complementary weighted dual adversarial network”. In: *Mechanical Systems and Signal Processing* 222 (2025), p. 111780.
- [20] Huaitao Xia et al. “Fault semantic knowledge transfer learning: Cross-domain compound fault diagnosis method under limited single fault samples”. In: *Reliability Engineering and System Safety* 260 (2025), p. 111050.
- [21] Ziyu Zhou and Wenhua Chen. “Cross-domain bearing fault diagnosis under missing labels with uncertainty modeling and digital twin correction”. In: *Measurement* 247 (2025), p. 116869.
- [22] Xin Li et al. “Multi-kernel weighted joint domain adaptation network for cross-condition fault diagnosis of rolling bearings”. In: *Reliability Engineering and System Safety* 261 (2025), p. 111109.
- [23] Jinyu Yang et al. “TVT: Transferable Vision Transformer for Unsupervised Domain Adaptation”. In: *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2023, pp. 520–530.

- [24] Ke Zhu and Jianxin Wu. “Residual Attention: A Simple but Effective Method for Multi-Label Recognition”. In: *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2021, pp. 184–193.
- [25] Xu Qin et al. “FFA-Net: Feature Fusion Attention Network for Single Image Dehazing”. In: *2020 AAAI Conference on Artificial Intelligence*. AAAI, 2020, pp. 11908–11915.
- [26] Biao Wang, Yong Qin, and Ao Ding. *Introduction to BJTU-RAO Bogie Datasets*. Tech. rep. Beijing, China: Beijing Jiaotong University, 2024.
- [27] Ruyi Huang, Yixiao Liao, and Shaohui Zhang. “Deep Decoupling Convolutional Neural Network for Intelligent Compound Fault Diagnosis”. In: *IEEE Access* 7 (2018), pp. 1848–1858.
- [28] Weihua Li, Hao Lan, and Junbin Chen. “WavCapsNet: An interpretable intelligent compound fault diagnosis method by backward tracking”. In: *IEEE Transactions on Instrumentation and Measurement* 72 (2023), pp. 1–11.
- [29] Ruyi Huang, Zhen Wang, and Jipu Li. “A Transferable Capsule Network for Decoupling Compound Fault of Machinery”. In: *2020 IEEE International Instrumentation and Measurement Technology Conference (I2MTC)*. IEEE, 2020, pp. 1–6.
- [30] Ruyi Huang, Jipu Li, and Yixiao Liao. “Deep Adversarial Capsule Network for Compound Fault Diagnosis of Machinery Toward Multidomain Generalization Task”. In: *IEEE Transactions on Instrumentation and Measurement* 70 (2021), pp. 1–11.
- [31] Chao Zhao, Enrico Zio, and Weiming Shen. “Domain Generalization for Cross-Domain Fault Diagnosis: An Application-Oriented Perspective and a Benchmark Study”. In: *Reliability Engineering & System Safety* 245 (2024), p. 109964.
- [32] Wentao Mao et al. “An Interpretable and Reliable Remaining Useful Life Prediction Approach Across Different Machines With Tensor Domain-Adversarial Regression Adaptation”. In: *IEEE Transactions on Reliability* 74.3 (2025), pp. 4076–4090.
- [33] Junsheng Xin et al. “Parameter-Adaptive Feature Mode Decomposition Method for Compound Fault Feature Extraction in Rotating Machinery”. In: *IEEE Sensors Journal* 25.10 (2025), pp. 17491–17502.
- [34] Yuan Wang et al. “Multimodal Correlation-Aware Fusion Framework for Enhanced Machinery Health Prognosis With Unlabeled and Low-Quality Data Exploitation”. In: *IEEE Transactions on Neural Networks and Learning Systems* 36.7 (2025), pp. 12040–12051.

Table A1: Diagnosis recall/precision/F1-score of Dashnosis on different fault classes on BJTU-RAO dataset (%).

Fault type	C1→C2	C1→C3	C1→C4	C2→C3	C2→C4	C3→C4
NC	100.00/99.56/99.78	99.12/100.00/99.56	100.00/100.00/100.00	100.00/100.00/100.00	100.00/100.00/100.00	100.00/99.12/99.56
IF	100.00/94.14/96.98	99.11/98.67/98.89	100.00/97.40/98.68	99.56/96.14/97.82	100.00/99.56/99.78	100.00/96.98/98.47
OF	100.00/99.56/99.78	100.00/99.56/99.78	100.00/99.56/99.78	100.00/97.83/98.90	99.56/100.00/99.78	99.56/96.55/98.03
RF	91.11/100.00/95.35	96.44/99.54/97.97	96.44/100.00/98.19	94.67/97.26/95.95	98.67/99.55/99.11	96.00/99.54/97.74
IOF	97.78/100.00/98.88	100.00/98.25/99.12	99.56/99.56/99.56	100.00/98.25/99.12	98.67/99.11/98.89	99.56/100.00/99.78
ORF	100.00/97.83/98.90	100.00/97.83/98.90	99.11/100.00/99.55	97.33/98.21/97.77	100.00/98.68/99.34	96.44/99.09/97.75
IOF	99.56/97.82/98.68	97.33/100.00/98.65	100.00/98.68/99.34	95.11/99.07/97.05	98.67/98.67/98.67	99.56/100.00/99.78

Table A2: Diagnosis recall/precision/F1-score of various methods on BJTU-RAO dataset (%).

Method	C1→C2	C1→C3	C1→C4	C2→C3	C2→C4	C3→C4
DDCNN	84.06/84.21/84.14	84.70/83.93/84.31	85.78/84.93/85.35	85.65/84.87/85.26	85.40/84.61/85.00	83.68/82.89/83.28
WavCapsNet	85.21/84.55/84.88	84.76/84.09/84.42	86.09/85.39/85.74	86.15/85.46/85.80	86.67/85.98/86.32	85.84/85.16/85.49
TCN	89.21/88.61/88.91	88.76/88.16/88.46	91.11/90.51/90.81	90.34/89.74/90.04	87.87/87.27/87.57	89.97/89.37/89.67
DACNs	92.25/91.77/92.01	91.75/91.23/91.51	93.71/93.27/93.48	93.27/92.87/93.08	92.19/91.69/91.91	93.02/92.51/92.78
DACNm	95.17/94.71/94.92	92.44/92.07/92.25	96.12/95.76/95.93	95.62/95.21/95.41	94.67/94.28/94.47	95.24/94.88/95.05
Dashnosis	98.35/98.41/98.33	98.98/99.00/98.98	99.30/99.31/99.30	98.10/98.11/98.09	99.37/99.37/99.36	98.73/98.75/98.73

Table A3: Diagnosis recall/precision/F1-score of Dashnosis on different fault classes on HUSTbearing dataset (%).

Fault type	C1→C2	C1→C3	C1→C4	C2→C3	C2→C4	C3→C4
NC	100.00/95.74/97.83	100.00/96.98/98.47	100.00/99.56/99.78	100.00/92.59/96.15	94.67/93.01/93.83	95.11/98.17/96.61
MI	96.44/100.00/98.19	98.67/98.23/98.45	100.00/95.74/97.83	96.44/100.00/98.19	100.00/100.00/100.00	100.00/100.00/100.00
SI	99.56/100.00/99.78	99.56/97.82/98.68	100.00/95.34/97.61	99.56/98.68/99.12	100.00/96.57/98.25	100.00/100.00/100.00
MO	100.00/100.00/100.00	100.00/100.00/100.00	100.00/100.00/100.00	100.00/100.00/100.00	95.11/100.00/97.49	100.00/97.83/98.90
SO	95.56/100.00/97.73	96.89/99.54/98.20	99.56/100.00/99.78	92.00/100.00/95.83	97.78/94.02/95.86	98.22/95.26/96.72
MC	100.00/96.57/98.25	98.67/98.67/98.67	98.22/100.00/99.10	100.00/97.40/98.68	100.00/100.00/100.00	97.78/100.00/98.88
SC	99.56/100.00/99.78	97.33/100.00/98.65	92.44/100.00/96.07	100.00/100.00/100.00	95.56/100.00/97.73	100.00/100.00/100.00

Table A4: Diagnosis recall/precision/F1-score of various methods on HUSTbearing dataset (%).

Method	C1→C2	C1→C3	C1→C4	C2→C3	C2→C4	C3→C4
DDCNN	83.82/83.68/83.75	84.54/84.66/84.60	85.52/85.43/85.47	85.92/85.98/85.95	85.12/85.26/85.19	83.95/83.84/83.89
WavCapsNet	85.47/85.33/85.40	84.70/84.82/84.76	86.35/86.26/86.30	86.33/86.39/86.36	86.90/87.04/86.97	85.60/85.49/85.54
TCN	88.77/88.63/88.70	88.94/89.06/89.00	90.77/90.68/90.72	90.69/90.75/90.72	87.66/87.80/87.73	90.12/90.01/90.06
DACNs	92.32/92.18/92.25	91.42/91.54/91.48	93.86/93.77/93.81	93.39/93.45/93.42	92.49/92.63/92.56	92.84/92.73/92.78
DACNm	94.93/94.79/94.86	92.75/92.87/92.81	95.94/95.85/95.89	95.89/95.95/95.92	94.33/94.47/94.40	95.44/95.33/95.38
Dashnosis	98.80/98.66/98.73	98.72/98.84/98.78	98.60/98.51/98.55	98.31/98.37/98.34	97.57/97.71/97.64	98.74/98.63/98.68